

NIVEAU 1 — LES TOUT PREMIERS PAS

- 1 Écrire à l'écran difficulté 1/10
- 2 Effacer ce qu'on a écrit difficulté 1/10
- 3 La plus petite cartouche difficulté 1/10
- 4 Écrire à plusieurs endroits difficulté 1/10
- 5 Une constante plutôt qu'un nombre difficulté 1/10
- 6 Réécrire au bout d'une seconde difficulté 1/10
- 7 Un nombre à l'écran difficulté 1/10
- 8 Effacer sans compter les lettres difficulté 1/10

NIVEAU 2 — RETENIR, ET RÉAGIR

- 9 Une variable, un octet difficulté 2/10
- 10 Lire un bouton difficulté 2/10
- 11 Un compteur qui déborde à 255 difficulté 2/10
- 12 Un dé, et le hasard du matériel difficulté 2/10
- 13 Deux mots de même longueur difficulté 2/10
- 14 Se souvenir de l'état d'avant difficulté 2/10
- 15 Un choix, puis plusieurs difficulté 2/10
- 16 Deux touches, deux effets difficulté 2/10

NIVEAU 3 — DESSINER

- 17 Dessiner sa propre tuile difficulté 3/10
- 18 Le dessin écrit là où on le pose difficulté 3/10
- 19 Intégrer une carte dans son projet difficulté 3/10
- 20 Les couleurs : palettes, variétés et thèmes difficulté 3/10
- 21 Poser tes tuiles à la souris difficulté 3/10
- 22 Poser une tuile difficulté 3/10
- 23 Quatre nuances, quatre signes difficulté 3/10
- 24 Une rangée, puis un mur difficulté 3/10
- 25 Un damier difficulté 3/10
- 26 Un lutin, hors de la grille difficulté 3/10

NIVEAU 4 — LE MOUVEMENT

- 27 Le front, ou pourquoi un appui compte trois fois difficulté 4/10
- 28 Un lutin, au pixel près difficulté 4/10
- 29 Un objet qui tombe difficulté 4/10
- 30 Un personnage de seize difficulté 4/10
- 31 Déplacer un lutin difficulté 4/10
- 32 Le garder dans l'écran difficulté 4/10
- 33 Une vitesse qui monte difficulté 4/10
- 34 Sauter, sans pesanteur difficulté 4/10
- 35 defiler(), et l'octet qui boucle difficulté 4/10

NIVEAU 5 — LE SON

- 36 Le son difficulté 5/10
- 37 Écrire une musique difficulté 5/10
- 38 Une note, au front difficulté 5/10
- 39 Une petite mélodie à la main difficulté 5/10
- 40 Un bruit difficulté 5/10
- 41 Une partition qui avance seule difficulté 5/10

- 42 Une musique qui reprend au bout difficulté 5/10

NIVEAU 6 — RANGER CE QU'ON MANIPULE

- 43 Un tableau, et une boucle « for » difficulté 6/10
- 44 Le panneau, et les nuances difficulté 6/10
- 45 Remplir un tableau, puis le dessiner difficulté 6/10
- 46 Une table gravée difficulté 6/10
- 47 Une grille difficulté 6/10
- 48 Chercher dans un tableau difficulté 6/10
- 49 Un texte qui porte un nom difficulté 6/10

NIVEAU 7 — DÉCOUPER SON PROGRAMME

- 50 Une fonction qui prend des arguments difficulté 7/10
- 51 Découper son programme difficulté 7/10
- 52 Nommer un geste qu'on refait difficulté 7/10
- 53 Une fonction qui rend une valeur difficulté 7/10
- 54 Un argument par défaut difficulté 7/10
- 55 Pourquoi une fonction ne s'appelle pas elle-même difficulté 7/10
- 56 #include, et les onglets de la page difficulté 7/10

NIVEAU 8 — CE QUI VA ENSEMBLE

- 57 Une « struct » : ce qui va ensemble difficulté 8/10
- 58 Le décor qui défile difficulté 8/10
- 59 La caméra : c'est le monde qui bouge difficulté 8/10
- 60 Trois variables qui parlent de la même chose difficulté 8/10
- 61 Un tableau de « struct » difficulté 8/10
- 62 Une méthode : la fonction qui connaît son objet difficulté 8/10
- 63 Les trois façons de faire la même chose difficulté 8/10
- 64 Une méthode qui en appelle une autre difficulté 8/10

NIVEAU 9 — UN VRAI JEU

- 65 Passer d'un écran à l'autre difficulté 9/10
- 66 La pesanteur et le saut difficulté 9/10
- 67 Le verre qui se brise difficulté 9/10
- 68 Un état de jeu difficulté 9/10
- 69 Le score, et le panneau difficulté 9/10
- 70 Des ennemis qui vivent et qui meurent difficulté 9/10
- 71 Les collisions difficulté 9/10
- 72 Ralentir sans compter les images à la main difficulté 9/10
- 73 Passer à la salle suivante, sans bouton difficulté 9/10

NIVEAU 10 — ALLER AU BOUT

- 74 Se souvenir d'une partie à l'autre difficulté 10/10
- 75 La musique d'un jeu difficulté 10/10
- 76 Le monde glisse, le héros ne bouge pas difficulté 10/10
- 77 Un record qui survit à l'extinction difficulté 10/10
- 78 Compter ce que coûte une boucle difficulté 10/10
- 79 Le même jeu, deux fois difficulté 10/10
- 80 Ce que le compilateur refuse, et pourquoi difficulté 10/10
- 81 Ce qu'une ligne de C++ devient, en vrai difficulté 10/10
- 82 Sinus, cosinus, une seule table difficulté 10/10

1. Écrire à l'écran

Une cartouche qui affiche un mot, et rien d'autre.

Le plus petit programme qui fait quelque chose. Comme tout programme C++, il commence à `int main()` — c'est là que la console arrive, et nulle part ailleurs.

`texte(colonne, ligne, "...")` écrit à partir d'une case : l'écran en fait **20 de large sur 18 de haut**, et la case (0, 0) est en haut à gauche.

La boucle `while (true)` n'est pas décorative. Un programme qui se termine laisse le processeur partir n'importe où ; ici, il tourne en attendant l'image suivante, soixante fois par seconde, et l'écran reste affiché.

LE PROGRAMME — 1308 OCTETS

```
int main() {
    texte(5, 6, "BONJOUR");
    texte(3, 9, "GAME BOY");

    while (true) {
        image();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

« BONJOUR » à la
colonne 5, ligne 6.

BONJOUR
GAME BOY

l'écran, au bout de vingt images

2. Effacer ce qu'on a écrit

Il n'y a pas de gomme : on écrit du vide par-dessus.

La carte de fond garde ce qu'on y a mis, tant qu'on ne le remplace pas. **L'espace est la tuile 0**, c'est-à-dire du vide : écrire des espaces par-dessus **est** l'effacement. C'est ce que fait ce programme, et c'est ce qu'il faut avoir compris une fois.

Le piège tient en un mot : il faut **autant d'espaces que de lettres**. `BONJOUR` en compte sept ; six espaces laisseraient un `R` tout seul au bout, et cette lettre orpheline est l'erreur la plus courante du débutant.

C'est pour cela que `effacer(6, 4, "BONJOUR")` **existe** : il ôte exactement la longueur du texte, comptée par le compilateur. Les deux lignes font la même chose, mais l'une ne peut pas se tromper d'un espace. La leçon « Effacer sans compter les lettres » y revient.

Pour effacer une zone plus large, `poser(colonne, ligne, 0)` fait la même chose case par case, et se met dans une boucle.

LE PROGRAMME — 1426 OCTETS

```
int main() {
    texte(6, 4, "BONJOUR");
    texte(3, 8, "APPUIE SUR A");

    while (true) {
        image();

        if (bouton(A)) {
            texte(6, 4, " "); // sept espaces, autant que BONJOUR
            texte(3, 8, "MERCI ");
        }

        if (bouton(B)) {
            texte(6, 4, "BONJOUR");
            texte(3, 8, "APPUIE SUR A");
        }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

A efface « BONJOUR » ; B le remet.

BONJOUR
APPUIE SUR A

l'écran, au bout de vingt images

3. La plus petite cartouche

Trois lignes, et la console affiche quelque chose.

Un programme C++ **ne s'exécute pas de haut en bas**. Il commence à `int main()`, et nulle part ailleurs — c'est la première chose qui change quand on vient d'un langage de script.

`texte(colonne, ligne, "...")` écrit à partir d'une case. L'écran en fait **20 de large sur 18 de haut**, et la case (0, 0) est en haut à gauche.

`while (true) { image(); }` n'est pas décoratif. Un programme qui se termine laisse le processeur partir n'importe où ; `image()` attend l'image suivante, et l'écran reste sur ce qu'on vient d'écrire.

LE PROGRAMME — 1287 OCTETS

```
int main() {  
    texte(5, 6, "BONJOUR");  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

« BONJOUR » à la
colonne 5, ligne 6.

BONJOUR

l'écran, au bout de vingt images

4. Écrire à plusieurs endroits

La position est un calcul comme un autre.

Trois appels, trois endroits. Rien de neuf — sauf ceci : la colonne et la ligne **ne sont pas obligées d'être écrites en clair**.

`texte(4 + 2, 8, "...")` marche, et `texte(x, y, "...")` avec des variables aussi. C'est ce qui rendra possible tout ce qui bouge.

LE PROGRAMME — 1338 OCTETS

```
int main() {
    texte(0, 0, "EN HAUT A GAUCHE");
    texte(4, 8, "AU MILIEU");
    texte(2, 17, "TOUT EN BAS");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois lignes, en haut, au milieu et tout en bas.

EN HAUT A GAUCHE

AU MILIEU

TOUT EN BAS

l'écran, au bout de vingt images

5. Une constante plutôt qu'un nombre

Nommer ne coûte rien du tout — et c'est la seule chose ici qui soit gratuite.

Un nombre écrit trois fois, ce sont **trois occasions de le changer à deux endroits**. Une `const` lui donne un nom.

Et elle **ne prend aucun octet de mémoire**. Elle est connue à la compilation : le compilateur remplace `COLONNE` par `5` dans le code machine, et il n'en reste rien dans la cartouche.

Comparer les deux compilations le montre — le nombre de variables annoncé ne bouge pas.

LE PROGRAMME — 1304 OCTETS

```
const uint8_t COLONNE = 5;
const uint8_t LIGNE = 6;

int main() {
    texte(COLONNE, LIGNE, "BONJOUR");
    texte(COLONNE, LIGNE + 2, "ENCORE");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« BONJOUR » puis «
ENCORE », deux lignes
plus bas.

BONJOUR
ENCORE

l'écran, au bout de vingt images

6. Réécrire au bout d'une seconde

Compter des images est l'horloge la plus simple qu'on ait.

La console affiche **soixante images par seconde**. Une boucle qui appelle `image()` soixante fois a donc laissé passer une seconde — sans minuteur, sans rien d'autre.

L'écran ne s'efface pas tout seul : pour ôter un mot, on écrit **autant d'espaces qu'il avait de lettres**. Un espace de moins, et la dernière lettre reste toute seule.

LE PROGRAMME — 1372 OCTETS

```
int main() {
    texte(4, 6, "PREMIER MOT");
    for (uint8_t i = 0; i < 60; i++) image();

    texte(4, 6, "          ");
    texte(4, 6, "SECOND MOT");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« PREMIER MOT » une
seconde, puis « SECOND
MOT » à sa place.

PREMIER MOT

l'écran, au bout de vingt images

7. Un nombre à l'écran

`texte` écrit des lettres ; `nombre` écrit un calcul.

`texte()` ne sait écrire que ce qui est entre guillemets, décidé à la compilation. **Un score qui change veut** `nombre()`.

`nombre(colonne, ligne, valeur)` écrit en base dix, sur **trois chiffres par défaut** — `042`. Un quatrième argument dit combien on en veut : `nombre(8, 4, vies, 1)` écrit `3` tout seul.

LE PROGRAMME — 1328 OCTETS

```
int main() {
    uint8_t vies = 3;
    uint8_t score = 42;

    texte(2, 4, "VIES");
    nombre(8, 4, vies, 1);

    texte(2, 6, "SCORE");
    nombre(8, 6, score);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« VIES 3 » et « SCORE 042 ».

V I E S 3
S C O R E 0 4 2

l'écran, au bout de vingt images

8. Effacer sans compter les lettres

Le compilateur connaît la longueur du texte. Autant qu'il la compte.

Écrire des espaces par-dessus **est** l'effacement — l'espace est la tuile 0. Mais il en faut **autant que de lettres** : un de moins laisse la dernière orpheline, un de plus mange la case d'à côté. Ni l'un ni l'autre n'est signalé, puisque écrire des espaces est parfaitement légal.

Or le compilateur **connaît** cette longueur : il la lit dans les guillemets, ou dans la table nommée. La compter soi-même, c'était se donner une occasion de se tromper que rien n'obligeait à prendre.

`effacer(colonne, ligne, quoi)` accepte les **trois** façons de dire combien : le texte lui-même, le nom d'un `const char`, ou un nombre — calculé s'il le faut. Et

`effacer(..., 0)` n'efface **aucune** case, ce qui n'allait pas de soi : sur un octet, décompter à partir de zéro donne 255.

`effacerPanneau()` fait de même sur le panneau — sans argument il le vide entièrement, à trois arguments il n'ôte qu'un mot.

LE PROGRAMME — 1410 OCTETS

```
const char FIN[] = "GAME OVER";

int main() {
    texte(2, 4, "BONJOUR");
    texte(2, 6, FIN);
    texte(2, 8, "ABCDEF");
    texte(2, 10, "JE RESTE");

    for (uint8_t i = 0; i < 90; i++) image();

    effacer(2, 4, "BONJOUR"); // sept cases, comptées pour nous
    effacer(2, 6, FIN);      // neuf, lues dans la table nommée
    uint8_t combien = 3;
    effacer(2, 8, combien);  // trois, décidées à l'exécution

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois lignes s'effacent au bout d'une seconde et demie ; « JE RESTE » ne bouge pas, et « DEF » survit.

```
BONJOUR
GAME OVER
ABCDEF
JE RESTE
```

l'écran, au bout de vingt images

9. Une variable, un octet

Retenir un nombre, et le montrer.

Une variable s'écrit `uint8_t` : un entier non signé de huit bits. C'est le seul nombre que cette machine connaisse — de 0 à 255, pas de virgule, pas de négatif. `int`, `char` et `auto` sont acceptés, et désignent la même chose ; `uint16_t`, `long` ou `float` sont **refusés avec leur numéro de ligne**, plutôt que traduits en douce sur un octet.

`attente++` est la façon C++ d'écrire `attente = attente + 1`. `+=`, `--`, `%=` existent aussi.

Pour afficher un chiffre calculé, on ne peut pas se servir de `texte()`, qui ne prend que des mots écrits en clair. On pose la **tuile** du chiffre : celle du 0 porte le numéro 27, et les neuf suivantes se suivent. D'où `27 + compte`.

LE PROGRAMME — 1378 OCTETS

```
int main() {
    uint8_t compte = 0;
    uint8_t attente = 0;

    texte(4, 4, "COMPTE");

    while (true) {
        image();

        attente++;

        if (attente > 30) {
            attente = 0;

            if (compte < 9) compte++;
        }

        poser(11, 4, 27 + compte);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Un chiffre qui monte de 0 à 9, une fois par demi-seconde.

COMPTE 0

l'écran, au bout de vingt images

10. Lire un bouton

Réagir à la manette.

`bouton(A)` rend 1 tant que le bouton est enfoncé. Les huit noms existent : `A B HAUT BAS GAUCHE DROITE START SELECT`.

Ici on lit le **niveau** : tant que A est tenu, le texte change. C'est ce qu'on veut pour courir ou viser — mais pas pour un menu : c'est la leçon « le front » qui règle ce cas.

`&&`, `||` et `!` s'écrivent comme en C++, et **s'arrêtent dès que la réponse est connue** : dans `i < n && t[i] == 0`, la case n'est pas lue si l'index est hors du tableau.

LE PROGRAMME — 1406 OCTETS

```
int main() {
    texte(3, 5, "APPUIE SUR A");

    while (true) {
        image();

        if (bouton(A) && !bouton(B)) {
            texte(3, 5, "BRAVO ");
        }

        if (bouton(B)) {
            texte(3, 5, "APPUIE SUR A");
        }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

« BRAVO » pendant qu'on tient A ; B remet le texte de départ.

APPUIE SUR A

l'écran, au bout de vingt images

11. Un compteur qui déborde à 255

Ce n'est pas un bogue : c'est ce que fait un octet.

`uint8_t` est **un octet** : de 0 à 255. À 255, `compteur++` ramène à 0 — et tout le projet est bâti là-dessus.

L'ordre compte. `image()` est **à la fin** : on dessine, puis on attend. Mettre l'attente en premier ferait sauter la première image.

Laisser tourner quelques secondes suffit à voir le compteur repasser par zéro.

LE PROGRAMME — 1289 OCTETS

```
int main() {
    uint8_t compteur = 0;

    while (true) {
        nombre(8, 8, compteur);
        compteur++;
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un nombre qui monte, et qui repart de 000 après 255.



008

l'écran, au bout de vingt images

12. Un dé, et le hasard du matériel

hasard() ne tire pas pile ou face : il lit un compteur qui tourne tout seul.

hasard() rend un octet imprévisible, de 0 à 255 — pas déjà réparti entre 1 et 6. Le reste d'une division, **hasard() % 6**, ramène ça entre 0 et 5 ; il suffit d'ajouter 1 pour une face de dé.

semer(images()) est appelé une seule fois, au tout début. Sans lui, la console partirait toujours du même point : son horloge interne n'a pas encore tourné à l'instant où le jeu démarre, et deux parties de suite tireraient la même suite.

Le dé ne se relance qu'au front de A, comme la leçon « Se souvenir de l'état d'avant » — sans lui, tenir A referait tourner le dé soixante fois par seconde, bien trop vite pour le lire.

LE PROGRAMME — 1466 OCTETS

```
uint8_t de = 1;
uint8_t avantA = 0;
uint8_t lances = 0;

uint8_t lancerDe() {
    return hasard() % 6 + 1;
}

void dessinerDe(uint8_t face) {
    effacer(8, 6, "0");
    nombre(8, 6, face, 1);
}

int main() {
    semer(images());

    texte(2, 2, "A: LANCER LE DE");
    texte(2, 4, "FACE:");
    texte(2, 9, "LANCES:");
    dessinerDe(de);

    while (true) {
        uint8_t a = bouton(A);

        if (a && !avantA) {
            de = lancerDe();
            dessinerDe(de);
            lances++;
            nombre(10, 9, lances);
        }
        avantA = a;

        image();
    }
}
```

CE QU'ON DOIT VOIR

Un dé de 1 à 6 sous FACE, qui change à chaque appui sur A ; LANCES compte les appuis.

```
A : LANCER LE DE
FACE :
      1
LANCES :
```

l'écran, au bout de vingt images

13. Deux mots de même longueur

Le piège du texte qui change : la fin de l'ancien reste.

bouton(A) rend 1 tant que le bouton est enfoncé, 0 sinon. Les huit noms sont A B HAUT BAS GAUCHE DROITE START SELECT .

Noter l'espace dans "APPUYE ". Sans lui, le mot fait six lettres et RELACHE en fait sept : le E final resterait affiché sous APPUYE , et l'on chercherait longtemps pourquoi.

Deux textes qui se remplacent doivent faire la même longueur. C'est une règle, pas une précaution.

LE PROGRAMME — 1324 OCTETS

```
int main() {
    while (true) {
        if (bouton(A)) texte(6, 8, "APPUYE ");
        else          texte(6, 8, "RELACHE");

        image();
    }
}
```

CE QU'ON DOIT VOIR

« RELACHE », et « APPUYE » tant qu'on tient A.

RELACHE

l'écran, au bout de vingt images

14. Se souvenir de l'état d'avant

Compter un appui, et non soixante par seconde.

Sans les deux lignes du milieu, `combien` monterait de **soixante par seconde** tant qu'on garde le doigt dessus : la boucle tourne soixante fois, et le bouton est enfoncé à chacun de ces tours.

Garder l'état d'avant, et ne compter que le **changement** — c'est ce qu'on appelle un front. On s'en sert pour tout ce qui doit arriver **une fois** : sauter, tirer, valider un menu.

La leçon « Le front, ou pourquoi un appui compte trois fois » montre ce qui arrive quand on l'oublie.

LE PROGRAMME — 1357 OCTETS

```
int main() {
    uint8_t combien = 0;
    uint8_t avant = 0;

    while (true) {
        uint8_t maintenant = bouton(A);

        /* Le FRONT : enfoncé maintenant, et pas au tour d'avant. */
        if (maintenant && !avant) combien++;
        avant = maintenant;

        texte(2, 6, "APPUIS");
        nombre(10, 6, combien);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le compteur monte de un par appui, même si l'on tient la touche.

APPUIS 000

l'écran, au bout de vingt images

15. Un choix, puis plusieurs

`&&` s'arrête dès que la réponse est connue.

`if ... else if ... else` enchaîne les cas. Rien que de très ordinaire — sauf la dernière ligne.

`bouton(B) && vies > 0` : si `bouton(B)` rend 0, **la partie droite n'est même pas calculée**. Ce n'est pas une finesse de compilation, c'est ce qui permet plus tard d'écrire

`if (i < n && table[i] == 0)` sans lire une case hors du tableau.

LE PROGRAMME — 1424 OCTETS

```
int main() {
    uint8_t vies = 3;

    while (true) {
        if (vies == 0)       texte(4, 8, "PERDU  ");
        else if (vies == 1)  texte(4, 8, "DERNIERE");
        else                 texte(4, 8, "CA VA  ");

        if (bouton(B) && vies > 0) vies--;
        image();
    }
}
```

CE QU'ON DOIT VOIR

« CA VA », puis «
DERNIERE », puis «
PERDU » — B après B.

CA VA

l'écran, au bout de vingt images

16. Deux touches, deux effets

Un `enum` donne des noms à des nombres, et `switch` les range côte à côte.

`TITRE` vaut 0, `JEU` vaut 1, `FIN` vaut 2 — mais **on ne l'écrit jamais**, et c'est tout l'intérêt : ajouter une scène au milieu ne renumérote rien à la main.

`Scene` devient un **type d'un octet**. `Scene ou = TITRE;` se relit mieux que `uint8_t ou = 0;`, et dit ce que la variable a le droit de contenir.

C'est la forme que prend presque tout jeu : quelques scènes, et un `switch` qui dit laquelle est en cours.

LE PROGRAMME — 1437 OCTETS

```
enum Scene { TITRE, JEU, FIN };

int main() {
    Scene ou = TITRE;

    while (true) {
        switch (ou) {
            case TITRE:
                texte(4, 8, "APPUIE START");
                if (bouton(START)) { texte(4, 8, " "); ou = JEU; }
                break;
            case JEU:
                texte(4, 8, "ON JOUE ");
                if (bouton(SELECT)) ou = FIN;
                break;
            case FIN:
                texte(4, 8, "C EST FINI ");
                break;
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

START passe au jeu,
SELECT y met fin.

APPUIE START

l'écran, au bout de vingt images

17. Dessiner sa propre tuile

Sortir des lettres — en chiffres, ou à la souris.

La police suffit à écrire, pas à jouer. Un **Tuile** prend **huit rangées de huit chiffres** : la nuance de chaque pixel, de 0 (le plus clair) à 3 (le plus sombre). Un **Perso** en prend seize sur seize.

Ce qui détermine le numéro d'une tuile, c'est l'ordre des déclarations. La police occupe les numéros 0 à 43 — les lettres, les chiffres, le carré plein. Le premier dessin du programme prend donc le **44**, le suivant le 45, et ainsi de suite. Le compilateur l'annonce à chaque fois, et l'atelier l'écrit sous la grille.

Le nom sert partout où un numéro est attendu : `poser(x, y, BLOC)` sur le fond, `sprite(n, x, y, BLOC)` en lutin, `lire(x, y) == BLOC` pour relire ce qui est affiché, et jusque dans une table de décor — `const uint8_t LIGNE[] = { BLOC, 0, BLOC };`. Il se calcule aussi : `BLOC + 1` est la tuile d'à côté. **Un Perso de seize, lui, compte pour quatre numéros** — c'est l'objet de la leçon suivante.

C'est exactement **pourquoi on écrit le nom et jamais le numéro**. Ajoute un **Perso** avant ta tuile, et elle passe de 44 à 48 : un `poser(2, 2, 44)` écrit à la main afficherait soudain autre chose, sans que rien ne le signale. Le nom, lui, reste juste — c'est le compilateur qui traduit, et c'est son travail.

Les deux façons de faire, et c'est la même chose. En code : change un **0** en **3** dans le programme, et le bloc change. À la souris : clique un pixel dans la grille ci-dessous. Le programme se réécrit tout seul sous tes yeux — c'est le même texte, écrit autrement.

Les chiffres ne sont pas la seule écriture. Huit chiffres à la file ne ressemblent pas à un dessin ; les signes `. - + #` disent exactement la même chose — le point est le vide, le dièse est le plein — et se relisent de loin : `"#.++++.#"`. Les deux donnent les mêmes octets. On n'en mélange pas deux dans une même tuile, et l'atelier rend à chacune la sienne quand tu peins.

C'est vrai partout dans ce projet : **le programme reste la seule vérité**. L'atelier ne garde aucun dessin de son côté ; il lit les `Tuile NOM = {...}` du texte, et il les réécrit. Un éditeur graphique qui tiendrait ses propres dessins finirait par ne plus dire la même chose que le code.

LE PROGRAMME — 1369 OCTETS

```
Tuile BLOC = {
  "33333333",
  "30000003",
  "30222203",
  "30222203",
  "30222203",
  "30222203",
  "30222203",
  "30000003",
  "33333333",
};

int main() {
  texte(4, 2, "MA TUILE");

  for (uint8_t x = 0; x < 8; x++) {
    poser(6 + x, 8, BLOC);
  }

  while (true) {
    image();
  }

  return 0;
}
```

CE QU'ON DOIT VOIR

Une rangée de huit blocs à cadre noir, au milieu de l'écran.

MA TUILE

l'écran, au bout de vingt images

La même chose, à la souris

BLOC

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BLOC

tuile n° 44 — poser(x, y, BLOC)

– zoom22×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

18. Le dessin écrit là où on le pose

Les huit rangées dans le « poser » lui-même — sans nom, et sans numéro.

Une case de l'écran, c'est **huit rangées de huit pixels**, jamais autre chose. La leçon d'avant leur donnait un nom — `Tuile BLOC = {...}` — puis posait ce nom. Mais un nom, à l'arrivée, **c'est un numéro** : `poser(2, 2, BLOC)` écrit 44 dans la carte de fond.

Ces huit rangées peuvent s'écrire **dans le `poser` lui-même**. Le compilateur grave la tuile, lui trouve son numéro, et l'écrit à la case demandée — il n'y a plus ni nom à inventer ni numéro à suivre, et le dessin est **là où il apparaît**.

Ce n'est pas un remplacement, c'est un choix. Ce qui sert PARTOUT mérite son nom : le sol ci-dessous est posé vingt fois, et `SOL` le dit à chaque ligne. Ce qui ne sert QU'À UN ENDROIT n'apprend rien de plus en s'appelant `TUILE3`, et oblige à lire deux endroits pour en comprendre un seul.

Le même dessin ne se grave qu'une fois. Écris deux fois le même nuage à dix lignes d'écart : le compilateur reconnaît qu'il l'a déjà, et ne dépense qu'une tuile. Il reconnaît aussi un dessin déjà déclaré en `Tuile`, **à travers les deux écritures** — `"21111112"` et `"+-----"` sont les mêmes pixels. La console n'en tient que 256 : les gaspiller ne se verrait qu'au jour où il n'y en aurait plus.

Cela marche aux quatre endroits où un numéro de tuile est attendu : `poser`, `poserPanneau`, `sprite`, et `sprite16` — qui veut alors **seize rangées de seize signes**, et les découpe lui-même en quatre tuiles. Ailleurs, des accolades restent une faute, et le compilateur le dit avec sa ligne.

Une chose que le dessin sur place ne fait pas : **il n'apparaît pas dans l'atelier**. Celui-ci lit les `Tuile NOM = {...}` du texte, et il ne saurait pas où réécrire un dessin qui n'a pas de nom. Ce qu'on veut peindre à la souris, on le nomme.

LE PROGRAMME — 1436 OCTETS

```
Tuile SOL = {
  "22222222",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "22222222",
};

int main() {
  texte(4, 2, "SUR PLACE");

  // Le sol sert partout : il garde son nom.
  for (uint8_t x = 0; x < 20; x++) {
    poser(x, 12, SOL);
  }

  // La porte ne sert qu'ici : ses huit rangées sont ici.
  poser(9, 11, {
    "#####",
    "#-----#",
    "#-####-#",
    "#-#-#-#-",
    "#-#-#-#-",
    "#-#-#-#-",
    "#-#-#-#-",
    "#-#-#-#-",
    "#####",
  });

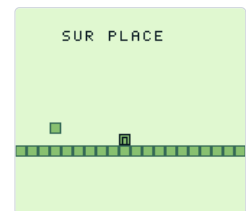
  // Le dessin de SOL, redit dans l'autre écriture : aucune tuile de plus.
  poser(3, 10, {
    "+++++++",
    "+-----+",
    "+-----+",
    "+-----+",
    "+-----+",
    "+-----+",
    "+-----+",
    "+++++++",
  });

  while (true) {
    image();
  }

  return 0;
}
```

CE QU'ON DOIT VOIR

Un sol de vingt cases, une porte posée au-dessus, et à gauche un pavé qui reprend le dessin du sol.



l'écran, au bout de vingt images

La même chose, à la souris

SOL

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

SOL

tuile n° 44 — poser(x, y, SO

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

19. Intégrer une carte dans son projet

Une carte dessinée à la souris est une fonction du programme : on l'appelle, et le décor apparaît.

Ce programme a les trois sortes de dessins du jeu : une **tuile de 8 × 8** (`Tuile ARBRE`), un **personnage de 16 × 16** (`Perso HEROS` — quatre tuiles d'un coup), et une **carte** (`VILLAGE` , tout un écran). Les deux premiers se peignent dans  **Les tuiles** ; la carte, dans  **La carte**. Essaie sur chacun les palettes, la sélection et le déplacement.

Une **carte**, c'est un décor entier dessiné à la souris, dans l'onglet  La carte. L'atelier ne la garde pas de son côté : il l'**écrit dans ton programme**, sous la forme d'une **fonction** — ici `void VILLAGE()`, entre deux commentaires `/* --- carte "VILLAGE" --- */` qui la marquent. C'est grâce à eux qu'il la retrouve et la réécrit quand tu dessines.

La créer : onglet  La carte, « + carte », et un nom en majuscules — VILLAGE, MAISON, NIVEAU1... Puis dessine : chaque carré de 8 × 8 que tu remplis devient une ligne `poser(colonne, ligne, {...})`, avec ses huit rangées. Un carré laissé vide n'est pas écrit : il ne coûte rien.

Lire le code de la carte : dans `poser(9, 10, {...})`, **9 est la colonne** (0 tout à gauche, 19 tout à droite) et **10 la ligne** (0 tout en haut, 17 tout en bas). Une case fait 8 × 8 pixels : la case (9, 10) commence donc au pixel x = 72, y = 80. Les **huit textes entre accolades** sont les huit rangées de pixels du carré, de haut en bas, et **chaque chiffre est un pixel** : 0 le plus clair, 1, 2, puis 3 le plus sombre. Regarde le toit, case (9, 9) : ses « 3 » dessinent la pente, rangée après rangée. En couleur, une ligne `teindre(colonne, ligne, palette)` dit en plus quelle palette prend la case.

Lire la carte pendant le jeu : `lire(colonne, ligne)` rend le **numéro de la tuile** affichée à cet endroit — 0 si la case est vide. C'est ainsi qu'un jeu sait si le joueur arrive sur un mur, sur le sol ou dans le vide. Le programme le montre en bas de l'écran : il relit la case (9, 11), du sol, et l'écrit avec `nombre()` ; la case (5, 11), vide, donne 0.

Une fonction ne fait rien tant qu'on ne l'appelle pas. C'est l'oubli le plus fréquent : la carte est dessinée, le programme compile, et l'écran reste vide. Il manque `VILLAGE();` dans `main`, **avant** la boucle du jeu — une seule fois : un décor reste à l'écran tant qu'on ne l'efface pas.

Essaie : mets `//` devant `VILLAGE();`. Le titre reste, la maison disparaît. Enlève-le : elle revient. Tout le décor tient dans cette seule ligne.

Plusieurs cartes : chacune a sa fonction. Pour passer de l'une à l'autre, on efface puis on appelle l'autre : `effacer();` puis `MAISON();`. Les huit carrés du sol, identiques, ne prennent **qu'une tuile** : le compilateur reconnaît un dessin qu'il a déjà.

Copier, déplacer : dans l'atelier de la carte, l'outil  **Sélection** choisit une zone — un rectangle **libre** tracé à la souris, ou d'un clic **un carré de 8 × 8** ou **un bloc de 16 × 16**, calés sur la grille. **Attrape** la zone et **glisse-la** : elle se déplace, et retombe sur la case la plus proche ; sa place redevient le fond. **Ctrl+C**, **Ctrl+X** et **Ctrl+V** la copient, la coupent et la collent. Dans l'atelier des tuiles,  **Sélectionner et déplacer** fait de même à l'intérieur d'une tuile de 8 × 8 ou d'un personnage de 16 × 16, et  **Copier le dessin**,  **Coller ici** et  **Dupliquer** copient un dessin entier. Chaque geste s'annule avec **Ctrl+Z**.

Un jeu, sans écrire la boucle : dans  La carte, « ► Jouer la scène » écrit `main` pour toi — la carte, les murs, le joueur qui bouge, les acteurs et les événements. Tes propres lignes de `main` sont alors remplacées ; l'atelier demande avant de le faire.

Poser une tuile, poser un personnage : `poser(6, 10, ARBRE);` met l'arbre dans la case (6, 10), et `teindre(6, 10, 1);` lui donne la palette 1 — celle de la forêt. `sprite16(0, 60, 72, HEROS);` pose le personnage **au pixel près**, en x = 60, y = 72 : un personnage n'est pas sur la grille, il se déplace librement par-dessus le décor.

À toi : descends à l'atelier de la carte, dessine un arbre à côté de la maison, et regarde un nouveau `poser(...)` apparaître dans `VILLAGE()`.

```
/* Une TUILE de 8 × 8 : un arbre. Elle se peint dans ■ Les tuiles. */
Tuile ARBRE = {
  "00111100",
  "01122110",
  "11222211",
  "12222221",
  "11222211",
  "01133110",
  "00033000",
  "00333300",
};

/* Un PERSONNAGE de 16 × 16 : quatre tuiles de 8 × 8 d'un coup. */
Perso HEROS = {
  "0000033333300000",
  "0000322222230000",
  "000322222223000",
  "0003211221123000",
  "0003211221123000",
  "000322222223000",
  "0000322332230000",
  "000003333300000",
  "000333333333000",
  "003222222222300",
  "0321222222221230",
  "0321222222221230",
  "000322222223000",
  "0003322003223000",
  "0003220000223000",
  "0033330000333300",
};

/* --- carte "VILLAGE" --- */
void VILLAGE() {
  poser(9, 9, {
    "00000033",
    "00003322",
    "00332222",
    "33222222",
    "32222222",
    "33333333",
    "31111113",
    "31111113",
  });
  poser(10, 9, {
    "33000000",
    "22330000",
    "22223300",
    "22222333",
    "22222223",
    "33333333",
    "00111113",
    "31111113",
  });
  poser(9, 10, {
    "31111113",
    "31133113",
    "31133113",
    "31111113",
    "31111333",
    "31111311",
    "31111311",
    "33333311",
  });
  poser(10, 10, {
    "31111113",
    "31111113",
    "33311113",
    "11311113",
    "11311113",
    "11311113",
    "11311113",
    "11333333",
  });
  poser(6, 11, {
    "33333333",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
  });
  poser(7, 11, {
    "33333333",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
  });
  poser(8, 11, {
    "33333333",
    "21212121",
    "12121212",
```

CE QU’ON DOIT VOIR

Le titre « MON VILLAGE », la maison sur huit carrés de sol, un arbre de chaque côté (dans la palette de la forêt), le personnage devant la maison, et en bas « SOL 053 » et « VIDE 000 ».



l'écran, au bout de vingt images

```

"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(9, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(10, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(11, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(12, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(13, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
}
/* --- fin de la carte "VILLAGE" --- */

int main() {
/* Les couleurs : la palette 1 pour l'arbre (une forêt). */
couleurFond(1, 0, 22, 31, 14);
couleurFond(1, 1, 10, 24, 6);
couleurFond(1, 2, 4, 14, 2);
couleurFond(1, 3, 12, 6, 2);

texte(4, 3, "MON VILLAGE");

VILLAGE(); // la carte : SANS CET APPEL, RIEN NE S'AFFICHE

/* La tuile ARBRE, posée à côté de la maison, dans la palette 1. */
poser(6, 10, ARBRE);
teindre(6, 10, 1);
poser(13, 10, ARBRE);
teindre(13, 10, 1);

/* Le personnage HEROS, au pixel près : un lutin de 16 x 16. */
sprite16(0, 60, 72, HEROS);

/* Relire la carte : le numéro de la tuile à une case, 0 si elle est vide. */
texte(1, 15, "SOL");
nombre(6, 15, lire(9, 11));
texte(1, 16, "VIDE");
nombre(6, 16, lire(3, 11));

while (true) {
image();
}

return 0;
}

```

20. Les couleurs : palettes, variétés et thèmes

Une tuile porte des numéros ; la palette de sa case leur donne leurs couleurs — la même tuile peut être de trois couleurs à la fois.

La règle de la console : un carré de 8×8 — une tuile — n'a que **4 couleurs**. Pas une de plus, même sur Game Boy Color : chaque carré prend **une palette** de 4 couleurs. Le carré d'à côté peut en prendre une autre : c'est ainsi qu'un écran entier affiche bien plus de 4 couleurs.

Le jeu a 8 palettes, numérotées de 0 à 7, et la **0 est la normale** (les verts de la Game Boy). Une tuile ne contient pas de couleurs : elle contient des **numéros**, de 0 à 3. C'est la palette de la case où on la pose qui dit de quelle couleur est chaque numéro.

Le programme le montre : la même tuile `BRIQUE`, posée trois fois. `teindre(colonne, ligne, palette)` donne sa palette à une case : la palette 1 (désert), la palette 2 (océan), et la dernière n'a pas de `teindre` — elle reste dans la palette 0, la normale. **Une seule tuile, trois couleurs** : c'est tout l'intérêt des palettes.

`couleurFond(palette, numéro, rouge, vert, bleu)` choisit une couleur : chaque composante va de 0 à 31. Chaque palette va du plus **clair** (numéro 0) au plus **sombre** (numéro 3) — le jeu garde ainsi une version lisible en quatre nuances.

Dans l'atelier, sans écrire de code : les 8 palettes sont en haut de l'atelier des tuiles et de la carte. Clique une couleur, peins : ce que tu peins **prend sa palette**. « 🎨 Changer cette couleur » change une couleur de la palette — partout où elle sert. « 🎨 Variétés » met dans une palette l'une des 11 palettes toutes prêtes (0 normale, campagne, bonbon, plage, forêt, glace, volcan, nuit, désert, océan, automne).

« 🎨 Thèmes » change d'un coup les 8 palettes du décor **et** les 8 des personnages, avec des couleurs **qui vont ensemble** : forêt, désert, glace, volcan, océan, hanté, bonbon — ou 0, la normale. Elles sont calculées : chaque palette part d'une teinte, et toutes descendent du clair au sombre par les mêmes quatre marches de lumière. **Ctrl+Z** revient en arrière.

Les personnages ont leurs 8 palettes à eux, de **3 couleurs + le transparent** (le numéro 0 ne s'affiche jamais : le décor se voit au travers). Un personnage de 16×16 est fait de 4 carrés : chacun peut avoir sa palette — jusqu'à 12 couleurs.

À toi : ajoute une quatrième brique en (10, 8) avec `teindre(10, 8, 1);` — elle prend les couleurs du désert. Puis, dans l'atelier, essaie « 🎨 Thèmes » et regarde les trois briques changer ensemble.

LE PROGRAMME — 1577 OCTETS

```
/* Une seule tuile : quatre NUMÉROS de couleur, de 0 (clair) à 3 (sombre). */
Tuile BRIQUE = {
    "33333333",
    "21112111",
    "21112111",
    "33333333",
    "11111112",
    "11121112",
    "33333333",
    "00000000",
};

int main() {
    /* La palette 1 : le désert — du plus clair au plus sombre. */
    couleurFond(1, 0, 31, 29, 22);
    couleurFond(1, 1, 28, 22, 10);
    couleurFond(1, 2, 20, 12, 4);
    couleurFond(1, 3, 8, 4, 2);
    /* La palette 2 : l'océan. */
    couleurFond(2, 0, 26, 31, 31);
    couleurFond(2, 1, 8, 24, 30);
    couleurFond(2, 2, 2, 12, 24);
    couleurFond(2, 3, 0, 3, 10);

    texte(3, 3, "UNE TUILE");
    texte(3, 4, "TROIS PALETTES");

    poser(4, 8, BRIQUE);
    teindre(4, 8, 1); // la case (4, 8) prend la palette 1 : le désert
    poser(6, 8, BRIQUE);
    teindre(6, 8, 2); // la même tuile, dans la palette 2 : l'océan
    poser(8, 8, BRIQUE); // pas de teindre : la palette 0, la normale

    while (true) {
        image();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Trois fois la même brique :
couleur sable (palette 1),
bleue (palette 2), et verte
(palette 0, la normale).

UNE TUILE
TROIS PALETTES

== == ==

l'écran, au bout de vingt images

La même chose, à la souris

BRIQUE

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BRIQUE

tuile n° 44 — poser(x, y, BRIQUE)

- zoom

22×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

21. Poser tes tuiles à la souris

Voir les carrés, tracer une zone — et le code s’écrit tout seul.

Jusqu’ici il fallait écrire `poser(6, 12, MUR);` à la main, case par case. Pour un sol de vingt cases, c’est vingt lignes, et une faute de frappe ne se voit pas.

Le **plan** ci-dessous montre les tuiles du programme telles qu’elles sont — des carrés, pas des numéros. On en choisit une, on **trace une zone** sur l’écran, et le programme reçoit ses `poser()`. La gomme remet du vide.

Le code écrit porte le **nom** de la tuile, jamais son numéro. C’est ce qui compte : un numéro devient faux le jour où l’on ajoute un dessin avant lui, alors qu’un nom reste juste. C’est le compilateur qui traduit, et c’est son travail.

L’outil ne possède que le bloc entre `// PLAN` et `// FIN DU PLAN`. Tout ce que tu écris ailleurs est à toi, et il n’y touche pas.

LE PROGRAMME — 1489 OCTETS

```
Tuile MUR = {
  "33333333",
  "32222223",
  "32122123",
  "32222223",
  "32221223",
  "32122223",
  "32222223",
  "33333333",
};

Tuile CAISSE = {
  "33333333",
  "31133113",
  "31133113",
  "33333333",
  "33333333",
  "31133113",
  "31133113",
  "33333333",
};

int main() {
  texte(4, 1, "TRACE UNE ZONE");

  // PLAN
  poser(0, 16, MUR);
  poser(1, 16, MUR);
  poser(2, 16, MUR);
  poser(3, 16, MUR);
  poser(9, 12, CAISSE);
  poser(10, 12, CAISSE);
  // FIN DU PLAN

  while (true) {
    image();
  }

  return 0;
}
```

CE QU’ON DOIT VOIR

Un bout de sol et deux caisses — puis ce que tu traces toi-même.



l’écran, au bout de vingt images

La même chose, à la souris



MUR



CAISSE



gomme



6 cases posées — le code s’écrit tout seul dans l’éditeur.

22. Poser une tuile

Une lettre à l'écran est une tuile comme une autre.

Les deux lignes affichent la même chose. `texte()` traduit des lettres en numéros de tuiles ; `poser(colonne, ligne, tuile)` pose le numéro directement.

La tuile 1 est le `A` de la police fournie, la 2 est un `B`. Il n'y a pas de magie : dessiner sa propre tuile, ce sera **ajouter un numéro à cette même suite**.

`lire(colonne, ligne)` fait l'inverse — il rend le numéro affiché à cet endroit. C'est ce qui servira pour les collisions.

LE PROGRAMME — 1397 OCTETS

```
int main() {
    texte(2, 4, "AVEC TEXTE ABC");

    texte(2, 8, "AVEC POSER");
    poser(14, 8, 1);
    poser(15, 8, 2);
    poser(16, 8, 3);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le même « ABC » sur les deux lignes.

```
AVEC TEXTE ABC
AVEC POSER ABC
```

l'écran, au bout de vingt images

23. Quatre nuances, quatre signes

Une tuile s’écrit en huit rangées de huit caractères.

. clair, **- moyen clair**, **+ moyen sombre**, **# sombre**. On peut aussi écrire **0123** — c’est la même chose.

Le nom **BRIQUE** devient un **numéro de tuile** : il s’écrit partout où l’on attend un numéro. Le compilateur grave les seize octets dans la cartouche et range la tuile à sa place au démarrage.

Dans la page, cette même tuile se dessine **à la souris** — l’atelier écrit exactement ces huit lignes dans le programme. Les deux chemins mènent au même endroit.

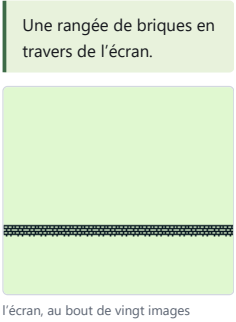
LE PROGRAMME — 1344 OCTETS

```
Tuile BRIQUE = {
    "#####",
    "#...#.",
    "#####",
    "..#...#",
    "#####",
    "#...#.",
    "#####",
    "..#...#",
};

int main() {
    for (uint8_t x = 0; x < 20; x++) poser(x, 12, BRIQUE);

    while (true) {
        image();
    }
}
```

CE QU’ON DOIT VOIR



La même chose, à la souris

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d’origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BRIQUE

tuile n° 44 — poser(x, y, BRIQUE)

- zoom22×

+ zoom

Plein écran

Peindre ✓

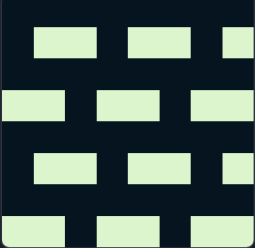
Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile



taille réelle — 8 × 8 pixels

24. Une rangée, puis un mur

Quatre-vingts appels en quatre lignes.

Écrire les quatre-vingts `poser()` à la main, ce sont quatre-vingts occasions de se tromper d'une case — et le jour où l'écran change de taille, il faut tout relire.

Deux boucles pour les bords horizontaux, deux pour les verticaux. Les coins sont posés deux fois : cela ne coûte rien, et l'écrire autrement demanderait quatre conditions.

LE PROGRAMME — 1473 OCTETS

```
Tuile MUR = {
    "#####",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#####",
};

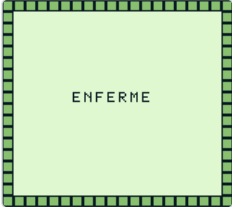
int main() {
    for (uint8_t x = 0; x < 20; x++) { poser(x, 0, MUR); poser(x, 17, MUR); }
    for (uint8_t y = 0; y < 18; y++) { poser(0, y, MUR); poser(19, y, MUR); }

    texte(6, 8, "ENFERME");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un cadre tout autour de l'écran, et « ENFERME » au milieu.



l'écran, au bout de vingt images

La même chose, à la souris

+ tuile 8 × 8

+ perso 16 × 16

 **Game Boy — 4 nuances**Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

MUR

tuile n° 44 — poser(x, y, MUR)

– zoom22×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile



taille réelle — 8 × 8 pixels

25. Un damier

Le reste d’une division, et le motif apparaît.

`(x + y) % 2` vaut 0 une case sur deux, en quinconce — c’est la définition même d’un damier, et elle tient en trois caractères.

`a ? b : c` choisit en une expression. On aurait pu écrire un `if` sur quatre lignes ; ici la ligne dit « pose ceci ou cela », et c’est ce qu’on veut lire.

LE PROGRAMME — 1423 OCTETS

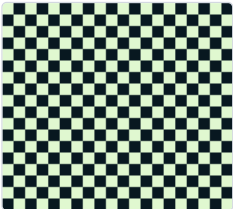
```
Tuile CLAIR = { ".....", ".....", ".....", ".....",
               ".....", ".....", ".....", "....." };
Tuile SOMBRE = { "#####", "#####", "#####", "#####",
                "#####", "#####", "#####", "#####" };

int main() {
    for (uint8_t y = 0; y < 18; y++) {
        for (uint8_t x = 0; x < 20; x++) {
            poser(x, y, (x + y) % 2 == 0 ? CLAIR : SOMBRE);
        }
    }

    while (true) {
        image();
    }
}
```

CE QU’ON DOIT VOIR

Un damier sur tout l’écran.



l’écran, au bout de vingt images

La même chose, à la souris

CLAIR

SOMBRE

+ tuile 8 x 8

+ perso 16 x 16

Game Boy — 4 nuances

Les quatre nuances de la console d’origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

CLAIR

tuile n° 44 — poser(x, y, CLAIR)

– zoom

22x

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 x 8 pixels

26. Un lutin, hors de la grille

Le fond est une grille de 8 en 8 ; un lutin ne l'est pas.

`sprite(numero, x, y, tuile)` pose un lutin **en pixels**, et non en cases : il peut se trouver à cheval entre deux tuiles du fond. C'est ce qui rend un mouvement doux possible.

Le premier argument est le **numéro du lutin**, de 0 à 39. C'est lui qu'on rappelle pour le déplacer, ou qu'on donne à `cache(n)` pour l'ôter.

LE PROGRAMME — 1328 OCTETS

```
Tuile HEROS = {
  ".....",
  ".#-#-#.",
  "#####",
  "#.####.",
  "#####",
  ".....",
  ".#...#.",
  "#####",
};

int main() {
  sprite(0, 76, 68, HEROS);
  texte(3, 14, "IL FLOTTE");

  while (true) {
    image();
  }
}
```

CE QU'ON DOIT VOIR

Un personnage au milieu, entre deux cases de la grille.



l'écran, au bout de vingt images

La même chose, à la souris



+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

HEROS

tuile n° 44 — poser(x, y, HERO

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile



taille réelle — 8 × 8 pixels

27. Le front, ou pourquoi un appui compte trois fois

Agir au moment précis où le bouton s'enfonce.

Un jeu tourne à **soixante images par seconde**. Un appui dure au moins cinq images : lu au niveau, il compte cinq fois. Sur un menu, on traverse trois lignes sans rien voir.

La réponse tient en une variable : l'état du bouton à l'image **précédente**. On agit quand il vaut 1 maintenant et valait 0 avant — ce qu'on appelle agir **au front**.

Le programme montre les deux compteurs côte à côte. Tiens A : celui du haut s'emballe, celui du bas monte d'un seul cran.

LE PROGRAMME — 1497 OCTETS

```
int main() {
    uint8_t aAvant = 0;
    uint8_t niveau = 0;
    uint8_t front = 0;

    texte(1, 4, "NIVEAU");
    texte(1, 8, "FRONT");

    while (true) {
        image();

        uint8_t a = bouton(A);

        if (a && niveau < 9) niveau++;
        if (a && !aAvant && front < 9) front++;

        aAvant = a;

        poser(10, 4, 27 + niveau);
        poser(10, 8, 27 + front);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Un appui sur A : le compteur « NIVEAU » saute, « FRONT » avance d'un.

N I V E A U 0

F R O N T 0

l'écran, au bout de vingt images

28. Un lutin, au pixel près

Ce qui sépare un jeu d'action d'un jeu de cases.

Le décor vit sur une grille de huit pixels. Un personnage, non : s'il ne pouvait s'arrêter que sur des multiples de huit, il ne sauterait pas, il **se téléporterait**.

`sprite(numero, x, y, tuile)` pose un carré de huit sur huit **où l'on veut**. Le matériel en a quarante, numérotés de 0 à 39.

Il n'y a rien à faire pour l'effacer : on le repose ailleurs à chaque image, et il a bougé.

LE PROGRAMME — 1550 OCTETS

```
Tuile BALLE = {
  "00333300",
  "03222230",
  "32222223",
  "32222223",
  "32222223",
  "32222223",
  "03222230",
  "00333300",
};

int main() {
  uint8_t x = 20;
  uint8_t y = 60;

  texte(3, 2, "CROIX POUR BOUGER");

  while (true) {
    image();

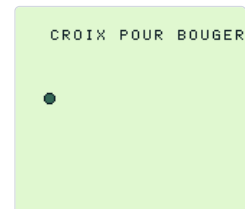
    if (bouton(DROITE) && x < 150) x += 2;
    if (bouton(GAUCHE) && x > 2) x -= 2;
    if (bouton(BAS) && y < 135) y += 2;
    if (bouton(HAUT) && y > 2) y -= 2;

    sprite(0, x, y, BALLE);
  }

  return 0;
}
```

CE QU'ON DOIT VOIR

Une boule qu'on déplace
à la croix, sans à-coups.



l'écran, au bout de vingt images

29. Un objet qui tombe

La pesanteur, en seizièmes de pixel.

Un objet qui tombe ne descend pas à vitesse constante : il **accélère**. Toute la physique tient en deux additions — la pesanteur s'ajoute à la vitesse, la vitesse s'ajoute à la position — et c'est de les faire **dans cet ordre**, à chaque image, qui fabrique la courbe d'une chute.

Reste un ennui : la console **ne connaît pas la virgule**. Au premier instant, la balle avance de deux seizièmes de pixel — arrondi, cela fait zéro, et elle ne partirait jamais.

On garde donc la position en **deux morceaux** : les pixels dans `y`, et les seizièmes de pixel dans `frac`. Seul `y` est montré ; `frac` est la mémoire de ce qui n'est pas encore un pixel entier, et qui le deviendra.

`frac >> 4` est une division par seize, `frac & 15` en est le reste. Le processeur ne sait pas diviser ; décaler de quatre bits, en revanche, lui coûte un seul tour d'horloge.

La chute dure **trois quarts de seconde**. Une démonstration qui ne se joue qu'une fois ne se voit pas : on compte donc les images passées au sol, et **au bout de quarante la balle est renvoyée en haut**. Elle tombe en boucle, et l'on peut la regarder tomber autant qu'on veut.

La relance remet **les trois** : la hauteur, les seizièmes et la vitesse. Le sol vient de les mettre à zéro une ligne plus haut — mais un départ qui compte sur le ménage fait par quelqu'un d'autre n'est pas un départ, et le jour où l'on relancera d'ailleurs la balle partira du ciel avec la vitesse qu'elle avait en s'écrasant.

`G` vaut 2, et ce nombre est le seul réglage : à 1 la balle flotte comme sur la Lune, à 6 elle tombe comme une enclume. C'est là, et nulle part ailleurs, que se décide ce qu'on ressent.

Et 9,81 dans tout ça ? Elle est dans `G`. Le processeur n'a ni virgule ni mètres : `G` se compte en seizièmes de pixel par image, par image, soit 450 pixels par seconde carrée à soixante images par seconde. Décidez qu'un mètre fait 46 pixels, et ces 450 pixels deviennent **9,81 m/s²** — l'écran mesure alors 3,10 m de haut, et la chute de 2,60 m dure les trois quarts de seconde que la vraie physique annonce. La pesanteur est la bonne ; c'est l'unité qui a changé.

LE PROGRAMME — 1510 OCTETS

```
Tuile BALLE = {
    "00333300",
    "03222230",
    "32222223",
    "32222223",
    "32222223",
    "32222223",
    "03222230",
    "00333300",
};

const uint8_t CIEL = 8; // le haut de la balle quand on la lâche
const uint8_t SOL = 128; // le haut de la balle quand elle touche terre
const uint8_t G = 2; // la pesanteur, en seizièmes de pixel par image
const uint8_t ATTENTE = 40; // les images passées au sol avant de recommencer

int main() {
    uint8_t y = CIEL; // les pixels – ce que l'écran montre
    uint8_t frac = 0; // les seizièmes – ce qu'il ne sait pas montrer
    uint8_t vy = 0; // la vitesse, en seizièmes de pixel par image
    uint8_t pose = 0; // depuis combien d'images elle est posée

    texte(4, 2, "CHUTE LIBRE");
    texte(2, 4, "ET CA RECOMMENCE");
    texte(0, 17, "#####");

    while (true) {
        image();

        vy = vy + G;

        frac = frac + vy;
        y = y + (frac >> 4);
        frac = frac & 15;

        if (y >= SOL) {
            y = SOL;
            frac = 0;
            vy = 0;
            pose = pose + 1;
        }

        if (pose > ATTENTE) {
            y = CIEL;
            frac = 0;
            vy = 0;
            pose = 0;
        }

        sprite(0, 76, y, BALLE);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

La balle part lentement, prend de la vitesse, s'arrête net sur le sol — puis remonte d'un coup et recommence.

CHUTE  LIBRE
ET CA RECOMMENCE



l'écran, au bout de vingt images

La même chose, à la souris



HEROS

+ tuile 8 × 8

+ perso 16 × 16

 **Game Boy — 4 nuances** Les quatre nuances de la console d'origine, et rien de plus.

nuance 3

Nuances

Passer en couleur

HEROS

tuiles n° 44 à 47 — `sprite16(n, x, y, HEROS)`

– zoom

11×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer le personnage





taille réelle — 16 × 16 pixels

31. Déplacer un lutin

Une position, quatre boutons — et surtout pas de front.

Ici l'on veut que ça bouge **tant que le doigt reste appuyé**. Pas de front, donc : le front sert à ce qui doit arriver une fois, pas à ce qui doit durer.

Savoir lequel des deux on veut est la moitié du travail sur cette machine.

LE PROGRAMME — 1408 OCTETS

```
Tuile HEROS = {
    "..####.", ".#-##-#.", "#####", "#.####.",
    "#####", "..#..#..", ".#...#.", "##...##",
};

int main() {
    uint8_t x = 76;
    uint8_t y = 68;

    while (true) {
        if (bouton(GAUCHE)) x--;
        if (bouton(DROITE)) x++;
        if (bouton(HAUT)) y--;
        if (bouton(BAS)) y++;

        sprite(0, x, y, HEROS);
        image();
    }
}
```

CE QU'ON DOIT VOIR

La croix déplace le personnage, pixel par pixel.



l'écran, au bout de vingt images

32. Le garder dans l'écran

Un octet ne prévient pas quand il repasse par zéro.

Sans le `x > GAUCHE_MAX`, arriver à 0 puis décrémenter donne **255** : le personnage réapparaît à droite. C'est le bogue le plus fréquent de cette machine, et il ne fait jamais planter — il téléporte, silencieusement.

Tester **avant** de bouger, et non corriger après. `if (x < 8) x = 8;` marche aussi, mais laisse la variable fausse pendant une ligne — et cette ligne finit toujours par en attirer d'autres.

LE PROGRAMME — 1416 OCTETS

```
Tuile HEROS = {
    ".####.", ".#-##-.", "#####", "#.###.#",
    "#####", "..#..#.", ".#...#.", "##....#",
};

const uint8_t GAUCHE_MAX = 8;
const uint8_t DROITE_MAX = 152;

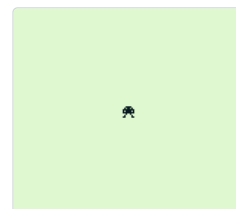
int main() {
    uint8_t x = 76;

    while (true) {
        if (bouton(GAUCHE) && x > GAUCHE_MAX) x--;
        if (bouton(DROITE) && x < DROITE_MAX) x++;

        sprite(0, x, 68, HEROS);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le personnage s'arrête
aux bords au lieu de
réapparaître en face.



l'écran, au bout de vingt images

33. Une vitesse qui monte

Accélérer, c'est une variable de plus.

Un objet qui accélère, c'est une **vitesse** qui monte. Tout ce qui suit — le saut, la pesanteur — n'est que cela, avec un signe en plus.

Elle ne montera pourtant **jamais jusqu'à 5** : plus le caillou va vite, plus tôt il touche le bas, et la chute suivante repart à 1. Le plafond écrit dans le code n'est pas celui qu'on observe — c'est le genre d'écart qu'on ne voit qu'en faisant tourner.

Pourquoi `depuis` **et non** `images() % 20 == 0`. Les deux paraissent équivalents, et ils ne le sont pas. Tester un multiple suppose que la boucle tourne **exactement une fois par image** ; dès que le corps s'alourdit, un tour franchit deux images d'un coup, le multiple est enjambé, et l'accélération n'arrive jamais.

Un compteur incrémenté à chaque tour et comparé avec `>=` ne peut pas rater son échéance. C'est la forme à prendre par défaut.

LE PROGRAMME — 1412 OCTETS

```
Tuile CAILLOU = {
    "..###..", ".#####", "#####", "#####",
    "#####", "#####", ".#####", ".#####",
};

int main() {
    uint8_t y = 0;
    uint8_t vitesse = 1;
    uint8_t depuis = 0;

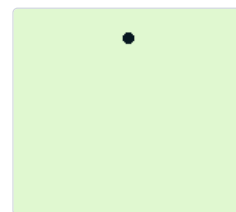
    while (true) {
        y += vitesse;
        if (y > 130) { y = 0; vitesse = 1; depuis = 0; }

        depuis++;
        if (depuis >= 20) { depuis = 0; if (vitesse < 5) vitesse++; }

        sprite(0, 76, y, CAILLOU);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un caillou qui tombe de plus en plus vite, et recommence.



l'écran, au bout de vingt images

34. Sauter, sans pesanteur

Compter les images qui restent à monter, plutôt qu'une vitesse signée.

Le front sert enfin : sans lui, garder **A** enfoncé relancerait le saut à chaque image, et le personnage collerait au plafond.

montee compte les images qui restent à monter. C'est plus simple qu'une vraie vitesse signée — et sur une machine **sans nombres négatifs**, plus simple veut dire plus juste.

La leçon « La pesanteur et le saut » reprend la même idée avec une vraie accélération.

LE PROGRAMME — 1458 OCTETS

```
Tuile HEROS = {
    "..####.", ".#-##-#.", "#####", "#.###.#",
    "#####", "..#.#..", ".#...#.", "##....##",
};

const uint8_t SOL = 120;

int main() {
    uint8_t y = SOL;
    uint8_t montee = 0;
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);

        /* On ne saute qu'au FRONT, et seulement depuis le sol. */
        if (appui && !avant && y == SOL) montee = 16;
        avant = appui;

        if (montee > 0) { y -= 2; montee--; }
        else if (y < SOL) y += 2;

        sprite(0, 76, y, HEROS);
        image();
    }
}
```

CE QU'ON DOIT VOIR

A fait sauter, et le personnage retombe au sol.



l'écran, au bout de vingt images

35. defiler(), et l'octet qui boucle

La carte fait 256 pixels — la même taille qu'un octet.

`defiler(x, y)` fait glisser tout le fond. La carte fait **256 pixels de côté** et revient toute seule à zéro.

`glisse` est un octet : il boucle exactement au bon moment, **sans qu'on ait rien à écrire**. C'est une coïncidence heureuse du matériel, et l'un des rares endroits où le débordement d'un octet est ce qu'on veut.

LE PROGRAMME — 1432 OCTETS

```
Tuile MOTIF = {
    "#-----", "-#-----", "--#-----", "---#-----",
    "----#---", "-----#--", "-----#-", "-----#",
};

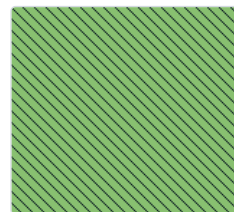
int main() {
    for (uint8_t y = 0; y < 18; y++)
        for (uint8_t x = 0; x < 20; x++)
            poser(x, y, MOTIF);

    uint8_t glisse = 0;

    while (true) {
        if (bouton(DROITE)) glisse++;
        if (bouton(GAUCHE)) glisse--;
        defiler(glisse, 0);
        image();
    }
}
```

CE QU'ON DOIT VOIR

DROITE et GAUCHE font
glisser le motif sans fin.



l'écran, au bout de vingt images

36. Le son

Quatre voix, dont trois qu'on peut jouer.

La console a une puce sonore depuis toujours, et rien ne lui parlait. `note(voix, hauteur, duree, volume)` joue une note sur la voix **1** ou **2** — deux signaux carrés — et `bruit(duree, volume)` frappe sur la quatrième, celle du bruit : une explosion, un pas, un tir.

Les hauteurs portent des noms : `D04`, `RED4`, `MI4` ... de `D02` à `SI6`, cinq octaves. Un nom se relit ; un numéro de fréquence, non. La table est calculée à la compilation et gravée dans la cartouche — 120 octets pour tout le clavier.

La **durée** se compte en 256^e de seconde, de 1 à 64. Une durée de `0` veut dire « sans fin » : la note tient jusqu'à `silence(voix)`. Le **volume** va de 0 à 15.

Deux voix, c'est une mélodie et son accompagnement. Ici la voix 2 double la voix 1 une octave plus bas — `MELODIE[pas] - 12`, puisqu'une octave fait douze demi-tons.

LE PROGRAMME — 1579 OCTETS

```
const uint8_t MELODIE[] = { D04, MI4, SOL4, D05, SOL4, MI4, RE4, SOL4 };

uint8_t pas = 0;
uint8_t attente = 0;

int main() {
    texte(3, 5, "A POUR UN BRUIT");

    while (true) {
        image();

        attente++;

        if (attente > 24) {
            attente = 0;

            note(1, MELODIE[pas], 20, 12);
            note(2, MELODIE[pas] - 12, 20, 6);

            pas++;
            if (pas >= sizeof(MELODIE)) pas = 0;
        }

        if (bouton(A)) bruit(6, 12);
        if (bouton(START)) { silence(1); silence(2); }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Une mélodie qui tourne en boucle ; A frappe un bruit sec.

A POUR UN BRUIT

l'écran, au bout de vingt images

37. Écrire une musique

Une mélodie qui joue toute seule, pendant que le jeu tourne.

Un `Air` s'écrit comme une tuile se dessine : une suite de pas, dans le programme. Chaque pas est une note et son volume — `"D04 12"` —, ou l'un des deux pas qui n'en sont pas : `"--"` fait taire la voix, `"=="` laisse la note d'avant continuer.

C'est `"=="` qui fait les notes longues. Sans lui, une blanche s'écrit en jouant la même note à chaque pas, et l'oreille entendrait quatre coups au lieu d'une note tenue.

`jouer(voix, AIR, vitesse)` le lance. La **vitesse** est le nombre d'images que dure un pas : huit images font un peu moins de huit pas par seconde. Un quatrième argument, `1`, le fait recommencer sans fin.

Ensuite, **le programme n'a plus rien à tenir**. L'air avance tout seul, soixante fois par seconde, dans l'interruption de la console — même si le jeu, lui, est en retard. Un tempo qui ralentirait avec la charge du jeu ne serait pas un tempo.

La **partition ci-dessous** écrit ces lignes à la souris, et le bouton « Écouter » la fait entendre sans même compiler. Les deux voix chantantes sont la 1 et la 2 ; la quatrième, celle du bruit, se frappe avec `bruit(6, 10)`.

LE PROGRAMME — 1853 OCTETS

```
Air THEME = {
    "D04 12", "==",      "MI4 12", "==",      "SOL4 12", "==",      "D05 13", "==",
    "SI4 11", "==",      "SOL4 11", "==",      "MI4 11", "==",      "--",      "--",
};

Air BASSE = {
    "D03 8",  "==",      "==",      "==",      "SOL3 8",  "==",      "==",      "==",
    "MI3 8",  "==",      "==",      "==",      "--",      "--",      "--",      "--",
};

uint8_t joue = 0;

int main() {
    texte(4, 5, "LA MUSIQUE");
    texte(1, 8, "A  ARRETER OU JOUER");

    jouer(1, THEME, 8, 1);
    jouer(2, BASSE, 8, 1);
    joue = 1;

    while (true) {
        image();

        if (bouton(A) && joue) {
            silence(1);
            silence(2);
            joue = 0;
        }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Rien de neuf à l'écran — mais deux voix qui jouent, et « A » qui les arrête.

LA MUSIQUE

A ARRETER OU JOUER

l'écran, au bout de vingt images

La même chose, à la souris

THEME

BASSE

+ air

THEME

Écouter

note

– silence

= tenir

▼

▲

– pas

+ pas

8

images par pas



`jouer(1, THEME, 8);` — le quatrième argument, 1, le fait recommencer sans fin.

Click la partition pour poser une note ; le bas de la grille règle le **volume** du pas.

Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

38. Une note, au front

Sans front, une note repart soixante fois par seconde.

`note(voix, hauteur, duree, volume)` : la voix est 1 ou 2, la durée se compte en images, le volume va de 0 à 15.

Les hauteurs s'écrivent `D04`, `RE4`, `MI4`... sur plusieurs octaves.

Le front est ici **indispensable** : sans lui, la note repartirait à chaque image et l'on n'entendrait qu'un grésillement continu.

LE PROGRAMME — 1399 OCTETS

```
int main() {
    texte(3, 8, "APPUIE SUR A");
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        if (appui && !avant) note(1, D04, 20, 12);
        avant = appui;
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un seul « bip » par appui, même en tenant la touche.

APPUIE SUR A

l'écran, au bout de vingt images

39. Une petite mélodie à la main

Une table gravée, et un pas toutes les vingt images.

`const uint8_t AIR[]` est **gravé dans la cartouche** : il ne prend aucun octet de mémoire de travail, et il ne peut pas changer. C'est exactement ce qu'on veut d'une mélodie.

`sizeof(AIR)` vaut 6, connu à la compilation. **Ajouter une note à la table suffit** — la boucle suit, sans qu'on touche à un nombre.

Ici le programme mène la mélodie et doit y penser à chaque tour. Le tutoriel suivant montre l'autre façon.

LE PROGRAMME — 1428 OCTETS

```
const uint8_t AIR[] = { D04, MI4, SOL4, D05, SOL4, MI4 };

int main() {
    uint8_t pas = 0;
    uint8_t depuis = 0;

    texte(2, 6, "NOTE");

    while (true) {
        depuis++;
        if (depuis >= 20) {
            depuis = 0;
            note(1, AIR[pas], 18, 12);
            nombre(9, 6, pas, 1);
            pas++;
            if (pas >= sizeof(AIR)) pas = 0;
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le numéro du pas monte de 0 à 5, puis recommence.

NOTE

l'écran, au bout de vingt images

40. Un bruit

La quatrième voix ne joue pas de notes.

`bruit(duree, volume)` frappe sur la voix du bruit — un pas, un tir, une porte qui claque.

Un troisième argument règle le **grain**, du sifflement aigu au choc sourd. C'est la seule voix qui n'ait pas de hauteur : il n'y a rien à accorder.

LE PROGRAMME — 1382 OCTETS

```
int main() {
    texte(4, 8, "A POUR TIRER");
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        if (appui && !avant) bruit(6, 10);
        avant = appui;
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un « tchac » sec à chaque appui sur A.

A POUR TIRER

l'écran, au bout de vingt images

41. Une partition qui avance seule

`jouer()` la lance, et le jeu fait autre chose pendant ce temps.

C'est toute la différence avec la mélodie à la main : là-bas le programme menait chaque note ; ici `jouer()` lance la partition et **elle avance toute seule**.

Un pas par élément. `"=="` tient la note d'avant, `"--"` fait un silence. Le troisième argument de `jouer()` est le nombre d'images par pas — le tempo.

Dans la page, cette partition s'écrit **à la souris**, sur une portée. L'atelier produit exactement ces lignes.

LE PROGRAMME — 1666 OCTETS

```
Air FANFARE = {
  "D04 12", "==", "MI4 12", "==", "SOL4 12", "==", "D05 12", "==", "==", "--",
};

int main() {
  jouer(1, FANFARE, 8);
  texte(4, 8, "CA JOUE");

  while (true) {
    image();
  }
}
```

CE QU'ON DOIT VOIR

Une fanfare qui monte, sans que le programme s'en occupe.



l'écran, au bout de vingt images

La même chose, à la souris

FANFARE

+ air

FANFARE

▶ Écouter

🎵 note

– silence

= tenir

▼

▲

– pas

+ pas

8

images par pas

`jouer(1, FANFARE, 8);` — le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas. Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

42. Une musique qui reprend au bout

`airFini()` dit quand la partition est arrivée à sa fin.

`airFini(voix)` rend 1 quand la voix a fini sa partition. On relance, et l'on compte les tours — c'est la boucle musicale la plus simple qui soit.

`silence(1)` couperait la voix net, si l'on voulait arrêter au milieu.

LE PROGRAMME — 1751 OCTETS

```
Air BOUCLE = {
    "D04 12", "=", "MI4 12", "=", "SOL4 12", "=", "MI4 12", "=",
};

int main() {
    jouer(1, BOUCLE, 10);
    uint8_t tours = 0;

    /* Écrit AVANT la boucle : sans cela, l'écran reste vide une seconde et
    demie, et l'on croit que le programme n'a pas démarré. */
    texte(2, 10, "TOURS");
    nombre(9, 10, tours);

    while (true) {
        if (airFini(1)) {
            jouer(1, BOUCLE, 10);
            tours++;
            nombre(9, 10, tours);
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le compteur de tours monte à chaque reprise.



l'écran, au bout de vingt images

La même chose, à la souris

BOUCLE

+ air

BOUCLE

▶ Écouter

🎵 note

– silence

= tenir

▼

▲

– pas

+ pas

8

images par pas

`jouer(1, BOUCLE, 8);` — le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas.

Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

43. Un tableau, et une boucle « for »

Ranger beaucoup de choses.

`uint8_t cases[20];` réserve vingt octets en mémoire de travail — de quoi tenir un puits, une carte, un inventaire. Ils vivent hors de la page rapide, qui ne contient que les variables.

Un tableau `const` est différent : il est **gravé dans la cartouche**. Il ne prend pas un octet de mémoire, et ne peut plus changer — le compilateur refuse d'y écrire. C'est ce qu'on veut pour une table de formes, un niveau, une courbe.

Le `for` de C++ tient sur une ligne ce que le `while` demandait en quatre, et **la variable du compteur n'existe que dans la boucle**. Deux boucles voisines peuvent toutes deux nommer leur compteur `i` sans se marcher dessus : c'est la portée du langage, et le compilateur la tient vraiment.

LE PROGRAMME — 1522 OCTETS

```
const uint8_t TABLE[] = { 1, 1, 2, 3, 5, 8, 3, 1, 4, 5 };

uint8_t cases[20];

int main() {
    texte(3, 3, "UN TABLEAU");
    texte(3, 11, "ET UNE TABLE");

    for (uint8_t i = 0; i < 20; i++) {
        cases[i] = i % 10;
    }

    for (uint8_t i = 0; i < 20; i++) {
        poser(i, 6, 27 + cases[i]);
    }

    for (uint8_t i = 0; i < 10; i++) {
        poser(i * 2, 14, 27 + TABLE[i]);
    }

    while (true) {
        image();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Les chiffres 0 à 9 deux fois, puis la table gravée, un chiffre sur deux.

```
UN TABLEAU
01234567890123456789

ET UNE TABLE
1 1 2 3 5 8 3 1 4 5
```

l'écran, au bout de vingt images

44. Le panneau, et les nuances

Un score qui reste en place pendant que le décor glisse.

Le décor défile ; le score, lui, ne doit pas bouger. Sans une seconde couche, il faudrait le redessiner à chaque image à la position que le défilement lui a donnée — et le moindre décalage se verrait.

Le **panneau** est cette couche : le matériel l'appelle la fenêtre, il la pose par-dessus le décor, et **elle ne défile pas**. `panneau(x, y)` la place, en pixels, et l'allume. On y écrit avec `textePanneau()` et `poserPanneau()`, exactement comme sur le fond.

Il n'y a que **deux cartes de fond** dans la console : le décor prend la première, le panneau la seconde. C'est un choix, et il vaut mieux le savoir que le découvrir.

`paletteFond(n0, n1, n2, n3)` choisit les quatre nuances, de la plus claire à la plus sombre — chaque nombre va de 0 à 3. Elles étaient posées une fois pour toutes à l'allumage ; ici on s'en sert pour assombrir le fond par paliers.

LE PROGRAMME — 1549 OCTETS

```
Tuile SOL = {
    "33333333",
    "32222223",
    "32122123",
    "32222223",
    "32221223",
    "32122223",
    "32222223",
    "33333333",
};

uint8_t d = 0;
uint8_t attente = 0;
uint8_t noirceur = 0;

int main() {
    for (uint8_t x = 0; x < 32; x++) {
        poser(x, 12, SOL);
        poser(x, 13, SOL);
    }

    textePanneau(1, 0, "SCORE 00");
    panneau(0, 128);

    while (true) {
        image();

        d++;
        defiler(d, 0);

        attente++;

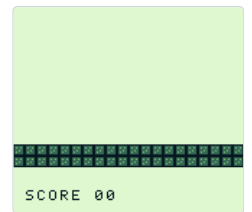
        if (attente > 40) {
            attente = 0;
            noirceur++;
            if (noirceur > 3) noirceur = 0;

            // le fond s'assombrit, puis revient
            paletteFond(noirceur, noirceur + 1, 2, 3);
        }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Le sol glisse ; « SCORE 00 »
» reste collé en bas, et les
nuances changent.



l'écran, au bout de vingt images

45. Remplir un tableau, puis le dessiner

Un tableau déclaré hors des fonctions est **VIDE** au démarrage.

Un **index est un octet : 256 cases au plus**. Ce n'est pas une limite arbitraire — c'est ce que la machine sait adresser d'un seul registre.

Un tableau déclaré hors des fonctions vit en mémoire de travail, et il **ne contient rien** au démarrage. Le remplir est le travail de la première boucle ; l'oublier donne un décor entièrement plat, sans qu'aucune erreur ne le dise.

LE PROGRAMME — 1430 OCTETS

```
uint8_t hauteurs[20];

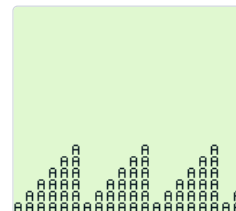
int main() {
    for (uint8_t i = 0; i < 20; i++) hauteurs[i] = i % 6;

    for (uint8_t i = 0; i < 20; i++)
        for (uint8_t h = 0; h <= hauteurs[i]; h++)
            poser(i, 17 - h, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Des colonnes en dents de scie, montant de 1 à 6 puis recommençant.



l'écran, au bout de vingt images

46. Une table gravée

Ce qui ne change jamais n'a rien à faire en mémoire.

`const uint8_t T[]` est **gravé dans la cartouche**, comme une tuile. Le compilateur l'annonce : ces octets sortent du décompte de la mémoire de travail.

Ôter le `const` la met en mémoire — et la rend modifiable, et la rend vide au démarrage. Les deux formes ont leur usage ; la différence se voit dans le compte rendu de compilation, et nulle part ailleurs.

LE PROGRAMME — 1358 OCTETS

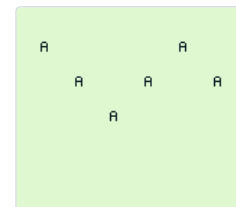
```
const uint8_t COLONNES[] = { 2, 5, 8, 11, 14, 17 };
const uint8_t LIGNES[]   = { 3, 6, 9, 6, 3, 6 };

int main() {
    for (uint8_t i = 0; i < sizeof(COLONNES); i++)
        poser(COLONNES[i], LIGNES[i], 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Six tuiles posées en zigzag, aux positions de la table.



l'écran, au bout de vingt images

47. Une grille

Deux index, et un calcul qu'on n'écrit pas.

`carte[1][c]` est rangé à plat, ligne après ligne, et le compilateur calcule `1 × LARGEUR + c`. C'est le calcul que tout le monde écrit à la main — et que tout le monde finit par écrire de travers le jour où la largeur change.

Le compilateur **exige les deux index** : `carte[3]` désignerait une rangée entière, et non un octet. Il le dit plutôt que de deviner.

LE PROGRAMME — 1590 OCTETS

```
const uint8_t LARGEUR = 10;
const uint8_t HAUTEUR = 8;

uint8_t carte[HAUTEUR][LARGEUR];

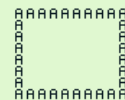
int main() {
    for (uint8_t l = 0; l < HAUTEUR; l++)
        for (uint8_t c = 0; c < LARGEUR; c++)
            carte[l][c] = (l == 0 || l == HAUTEUR - 1 || c == 0 || c == LARGEUR - 1) ? 1 : 0;

    for (uint8_t l = 0; l < HAUTEUR; l++)
        for (uint8_t c = 0; c < LARGEUR; c++)
            if (carte[l][c]) poser(c + 5, l + 5, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un rectangle creux, de dix sur huit, posé au milieu.



l'écran, au bout de vingt images

48. Chercher dans un tableau

Sortir dès qu'on a trouvé, et réserver une valeur pour « pas trouvé ».

`break` sort de la boucle ; `continue` passerait au tour suivant.

`ou = 255` sert de « pas trouvé ». Sur un octet il n'y a pas de valeur spéciale, alors **on en réserve une** — et l'on choisit celle qui ne peut pas être un index valide, puisqu'un tableau fait 256 cases au plus.

LE PROGRAMME — 1424 OCTETS

```
uint8_t valeurs[8];

int main() {
    for (uint8_t i = 0; i < 8; i++) valeurs[i] = i * 3;

    uint8_t cherche = 12;
    uint8_t ou = 255;

    for (uint8_t i = 0; i < 8; i++) {
        if (valeurs[i] == cherche) { ou = i; break; }
    }

    texte(2, 6, "TROUVE EN");
    nombre(13, 6, ou);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« TROUVE EN 004 » — 12
est bien la cinquième
valeur.

TROUVE EN 004

l'écran, au bout de vingt images

49. Un texte qui porte un nom

Le même message à deux endroits, écrit une fois.

`const char NOM[] = "..."` est gravé dans la cartouche, comme une table. Il s'écrit ensuite **partout où `texte()` attend des guillemets**.

Le même message répété trois fois entre guillemets, ce sont trois occasions de le changer à deux endroits. Ici il n'y en a qu'un.

LE PROGRAMME — 1300 OCTETS

```
const char TITRE[] = "GAME OVER";

int main() {
    texte(5, 6, TITRE);
    texte(5, 12, TITRE);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« GAME OVER » deux fois,
écrit une seule fois dans
le programme.

GAME OVER

GAME OVER

l'écran, au bout de vingt images

50. Une fonction qui prend des arguments

Écrire une fois ce qu'on fera dix fois.

Voici ce que le C++ apporte de plus visible. Une fonction prend des **arguments**, et rend une **valeur** : `uint8_t distance(uint8_t a, uint8_t b)`. On l'écrit une fois, on l'appelle partout, et l'endroit où l'on écrit `barre(2, 6, 5)` dit exactement ce qui va se passer.

Une fonction qui ne rend rien s'écrit `void`. Une fonction qui annonce rendre un `uint8_t` **doit** le faire : si aucun `return` ne rend de valeur, le compilateur refuse le fichier, plutôt que de laisser la fonction rendre ce qui traînait dans le processeur.

Un prix à payer, et il est dit franchement : **une fonction ne peut pas s'appeler elle-même**. Les arguments vivent à une place fixe, réservée une fois pour toutes — faute d'une pile praticable sur cette machine. Un appel imbriqué écraserait les arguments de l'appel en cours ; le compilateur nomme le cycle et s'arrête là.

LE PROGRAMME — 1471 OCTETS

```
Tuile BLOC = {
    "33333333",
    "30000003",
    "30222203",
    "30222203",
    "30222203",
    "30222203",
    "30000003",
    "33333333",
};

// Une fonction qui rend une valeur.
uint8_t distance(uint8_t a, uint8_t b) {
    if (a > b) return a - b;
    return b - a;
}

// Une fonction qui n'en rend aucune : « void ».
void barre(uint8_t colonne, uint8_t ligne, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, BLOC);
    }
}

int main() {
    texte(2, 2, "DES ARGUMENTS");

    barre(2, 6, 5);
    barre(2, 8, 12);
    barre(2, 10, distance(3, 20));

    while (true) {
        image();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Trois barres de longueurs différentes : 5, 12, puis 17 cases.

DES ARGUMENTS

```
#####
#####
#####
```

l'écran, au bout de vingt images

51. Découper son programme

Le même travail, écrit pour être relu dans trois jours.

Tout ce que fait ce programme tiendrait dans `main()`. Ce serait plus court à taper — et illisible dès qu'il grandit. Une boucle de jeu de deux cents lignes, on ne la relit pas : on la contourne, on lui ajoute une rustine à côté, et le désordre s'installe.

Une fonction est d'abord un nom. `dessinerLeSol()` dit ce qu'elle fait ; le lecteur n'a pas besoin d'ouvrir la boucle qui pose vingt tuiles pour le savoir. Une fonction sans argument ni retour reste utile — c'est même là qu'elle sert le plus.

Un nombre écrit en clair au milieu du code ne dit rien. `poser(colonne, 14, HERBE)` : pourquoi 14 ? `const uint8_t LIGNE_DU_SOL = 14;` répond, et le jour où le sol descend, il n'y a qu'un seul endroit à changer. Une `const` ne coûte **aucun octet** : sa valeur est connue à la compilation.

`return;` **au milieu d'une fonction** évite d'imbriquer trois `if`. `fairePasserLeTemps()` s'arrête tout de suite quand ce n'est pas encore l'heure, et la suite se lit à plat.

Un programme peut tenir sur plusieurs fichiers. `#include "dessins.cpp"` verse un autre fichier à cet endroit, avant toute compilation — et une faute y est signalée **avec le nom de son vrai fichier**, pas avec la ligne du texte collé. C'est ainsi qu'est écrit `exemples/decoupe/` : cinq fichiers, une seule cartouche, au dernier octet près.

Dans l'atelier, les fichiers sont les **onglets sous le titre** : « + fichier » en ajoute un, et il suffit de l'inclure. L'exemple « Un programme sur CINQ fichiers » les ouvre tous les cinq d'un coup, pour voir à quoi ça ressemble.

```

/*
 * Le même décor, écrit pour être RELU.
 *
 * Tout ce qui suit tiendrait dans main(). Ce serait plus court à taper, et
 * illisible trois jours plus tard.
 */

const uint8_t LIGNE_DU_SOL = 14;
const uint8_t LIGNE_DU_CIEL = 3;
const uint8_t COLONNES = 20;

Tuile HERBE = {
    "01100110",
    "11111111",
    "22222222",
    "22322322",
    "22222222",
    "23223223",
    "22222222",
    "33333333",
};

Tuile NUAGE = {
    "00000000",
    "00011100",
    "00111110",
    "01111111",
    "11111111",
    "01111110",
    "00000000",
    "00000000",
};

uint8_t nuage = 3;
uint8_t attente = 0;

/* Chaque fonction porte un nom qui DIT ce qu'elle fait. */
void ecrireLeTitre() {
    texte(5, 1, "MON DECOR");
}

void dessinerLeSol() {
    for (uint8_t colonne = 0; colonne < COLONNES; colonne++) {
        poser(colonne, LIGNE_DU_SOL, HERBE);
    }
}

void effacerLeCiel() {
    for (uint8_t colonne = 0; colonne < COLONNES; colonne++) {
        poser(colonne, LIGNE_DU_CIEL, 0);
    }
}

void dessinerLeCiel() {
    effacerLeCiel();
    poser(nuage, LIGNE_DU_CIEL, NUAGE);
}

void fairePasserLeTemps() {
    attente++;
    if (attente < 20) return;

    attente = 0;
    nuage++;
    if (nuage >= COLONNES) nuage = 0;
    dessinerLeCiel();
}

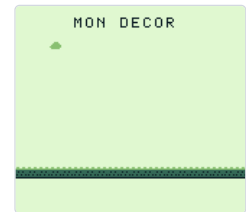
int main() {
    ecrireLeTitre();
    dessinerLeSol();
    dessinerLeCiel();

    while (true) {
        image();
        fairePasserLeTemps();
    }

    return 0;
}

```

Un titre, un sol d'herbe
sur toute la largeur, et un
nuage qui traverse le ciel.



l'écran, au bout de vingt images

La même chose, à la souris



HERBE



NUAGE

+ tuile 8 × 8

+ perso 16 × 16

 **Game Boy — 4 nuances**Les quatre nuances de la console d'origine, et rien de plus.

 Passer en couleur

nuance 3

Nuances

HERBE

tuile n° 44 — poser(x, y, HERB

- zoom

22×

+ zoom

Plein écran

Peindre ✓

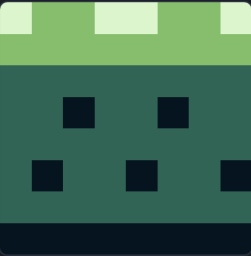
Sélectionner et déplacer

 Copier le dessin

 Coller ici

 Dupliquer

 Effacer la tuile



taille réelle — 8 × 8 pixels

52. Nommer un geste qu'on refait

Une fonction avec un argument, et trois appels.

Les fonctions prennent des arguments, et c'est là tout l'intérêt. Sans argument, il faudrait trois fonctions, ou une variable globale à poser avant chaque appel.

`void` dit qu'elle ne rend rien. Le compilateur le vérifie : une fonction qui annonce rendre un `uint8_t` sans jamais faire `return` est refusée, avec sa ligne.

LE PROGRAMME — 1366 OCTETS

```
Tuile BRIQUE = {
    "#####", "#..#..#", "#####", "..#..#",
    "#####", "#..#..#", "#####", "..#..#",
};

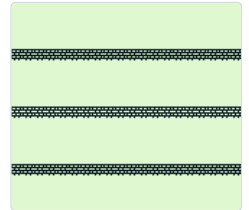
void rangee(uint8_t ligne) {
    for (uint8_t x = 0; x < 20; x++) poser(x, ligne, BRIQUE);
}

int main() {
    rangee(4);
    rangee(9);
    rangee(14);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois rangées de briques, écrites par un seul bout de code.



l'écran, au bout de vingt images

La même chose, à la souris

 + tuile 8 × 8 + perso 16 × 16

 **Game Boy — 4 nuances** Les quatre nuances de la console d'origine, et rien de plus.  Passer en couleur

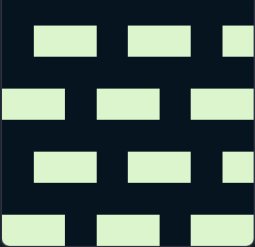
nuance 3

Nuances 

BRIQUE tuile n° 44 — poser(x, y, BRIQUE)

 zoom 22×   Plein écran  Peindre ✓  Sélectionner et déplacer  Copier le dessin  Coller ici

 Dupliquer  Effacer la tuile

 taille réelle — 8 × 8 pixels

53. Une fonction qui rend une valeur

Ce qui entre, ce qui sort — et ce qu'on ne réécrit pas deux fois.

Les deux appels rendent 70. Sans la fonction, il faudrait écrire le `if` deux fois — et le jour où l'on veut borner l'écart à 60, le corriger aux deux endroits.

Le résultat s'utilise dans un calcul : `nombre(10, 6, distance(a, b) * 2)` marche, comme n'importe quelle expression.

LE PROGRAMME — 1361 OCTETS

```
uint8_t distance(uint8_t a, uint8_t b) {
    if (a > b) return a - b;
    return b - a;
}

int main() {
    texte(2, 6, "ECART");
    nombre(10, 6, distance(30, 100));
    nombre(10, 8, distance(100, 30));

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

070 deux fois — l'ordre des arguments ne change rien.

```
ECART    070
          070
```

l'écran, au bout de vingt images

54. Un argument par défaut

Ce qui n'est pas écrit prend la valeur annoncée.

L'argument par défaut est rempli à l'appel, et non dans la fonction — comme en C++. Cela lui permet de dépendre de l'endroit d'où l'on appelle.

Une règle : un argument sans valeur par défaut **ne peut pas suivre** un argument qui en a une. Sinon, à `barre(4, 20)`, on ne pourrait plus deviner lequel manque. Le compilateur le refuse plutôt que de choisir à notre place.

LE PROGRAMME — 1374 OCTETS

```
void barre(uint8_t ligne, uint8_t combien, uint8_t tuile = 1) {
    for (uint8_t x = 0; x < combien; x++) poser(x, ligne, tuile);
}

int main() {
    barre(4, 20);
    barre(8, 10);
    barre(12, 5, 2);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois barres de longueurs différentes ; la dernière d'une autre tuile.

```
AAAAAAAAAAAAAAAAAAAAA
AAAAAAAAA
BBBBB
```

l'écran, au bout de vingt images

55. Pourquoi une fonction ne s'appelle pas elle-même

Le refus le plus important du projet — et comment dérouler la boucle.

Écrire `return n + somme(n - 1);` est **refusé**, avec le cycle nommé : « `somme` finit par s'appeler elle-même (somme → somme) ».

Les arguments vivent à une adresse fixe, réservée une fois pour toutes — c'est ce que fait tout compilateur C d'une machine huit bits, faute d'une pile praticable. Un appel imbriqué écraserait les arguments de l'appel en cours, et la fonction reprendrait son travail avec les valeurs de l'autre.

C'est un bogue silencieux, et indébogable. Le compilateur le nomme avant qu'il n'arrive. Voici la même chose, déroulée avec `while`.

LE PROGRAMME — 1354 OCTETS

```
uint8_t somme(uint8_t n) {
    uint8_t total = 0;
    while (n > 0) { total += n; n--; }
    return total;
}

int main() {
    texte(2, 6, "SOMME DE 1 A 5");
    nombre(8, 8, somme(5));

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

015 — la somme de 1 à 5, calculée sans récursion.

SOMME DE 1 A 5
015

l'écran, au bout de vingt images

56. #include, et les onglets de la page

Le fichier voisin est versé ici, avant toute compilation.

`#include "dessins.cpp"` **verse le fichier voisin à cet endroit** avant que le compilateur ne voie quoi que ce soit. Il n'y a donc ni éditeur de liens, ni ordre de compilation à connaître.

Dans la page, les fichiers voisins sont des **onglets** : `#include` ne voit pas la différence. Et quand une faute tombe dans un fichier inclus, le message désigne **son** fichier et **sa** ligne — pas la ligne du texte assemblé, que personne n'a jamais vue.

Le programme ci-dessous tient en un fichier ; la leçon « Découper son programme » en montre un sur cinq.

LE PROGRAMME — 1312 OCTETS

```
uint8_t doubler(uint8_t n) { return n * 2; }

int main() {
    texte(2, 6, "DOUBLE DE 21");
    nombre(8, 8, doubler(21));

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« DOUBLE DE 21 », et 042
en dessous — la fonction
a été appelée.

DOUBLE DE 21
042

l'écran, au bout de vingt images

57. Une « struct » : ce qui va ensemble

Une troupe d'ennemis, et non six tableaux parallèles.

Un ennemi a une position, une vitesse, une santé. Sans `struct`, cela fait trois tableaux qu'il faut penser à garder alignés — et le jour où l'on en trie un sans trier les autres, les vies changent de propriétaire.

Une `struct` range ces champs ensemble. `Etoile etoiles[4];` réserve quatre enregistrements ; `etoiles[i].x` en désigne un champ. Le compilateur calcule le décalage lui-même, et `sizeof(Etoile)` dit ce qu'un enregistrement pèse.

Le numéro d'un lutin peut être **calculé** : `sprite(i, ...)` dans une boucle affiche toute la troupe, chacun avec ses valeurs à lui. C'est ce qui rend la `struct` utile plutôt que décorative.

Une `struct` peut en contenir une autre : `struct Ennemi { Point ou; uint8_t vie; };`, et l'on écrit `troupe[i].ou.x`. Le compilateur additionne les décalages lui-même.

Ce qu'elle ne sait pas faire : s'affecter d'un bloc, se passer en argument, se rendre par une fonction. On nomme le champ qu'on veut lire ou écrire — et le compilateur le dit plutôt que de traduire à peu près.

LE PROGRAMME — 1686 OCTETS

```
Tuile CORPS = {
    "00333300",
    "03222230",
    "32222223",
    "32122123",
    "32222223",
    "32211223",
    "03222230",
    "00333300",
};

struct Etoile {
    uint8_t x;
    uint8_t y;
    uint8_t vitesse;
};

const uint8_t COMBIEN = 4;

Etoile etoiles[COMBIEN];

void placer() {
    for (uint8_t i = 0; i < COMBIEN; i++) {
        etoiles[i].x = 20 + i * 30;
        etoiles[i].y = 32 + i * 24;
        etoiles[i].vitesse = 1 + i;
    }
}

void avancerToutes() {
    for (uint8_t i = 0; i < COMBIEN; i++) {
        etoiles[i].x += etoiles[i].vitesse;

        if (etoiles[i].x > 152) etoiles[i].x = 0;

        sprite(i, etoiles[i].x, etoiles[i].y, CORPS);
    }
}

int main() {
    texte(2, 1, "UNE TROUPE");

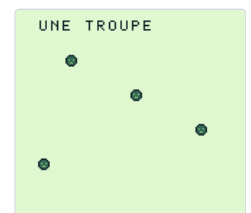
    placer();

    while (true) {
        image();
        avancerToutes();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Quatre boules qui traversent l'écran, chacune à sa vitesse.



l'écran, au bout de vingt images

58. Le décor qui défile

Un monde plus large que l'écran.

La carte de fond fait **32 cases sur 32**, soit 256 pixels de côté ; l'écran n'en montre que 20 sur 18. Le reste attend hors-champ.

`defiler(x, y)` fait glisser la fenêtre. Au-delà de 255, le compte revient tout seul à zéro : le décor est un **ruban sans fin**.

C'est ce qui permet un niveau aussi long qu'on veut : quand la caméra franchit une case, on redessine la colonne qui va entrer par la droite, à la place de celle qui vient de sortir par la gauche.

LE PROGRAMME — 1459 OCTETS

```
Tuile MUR = {
    "33333333",
    "32222223",
    "32122123",
    "32222223",
    "32221223",
    "32122223",
    "32222223",
    "33333333",
};

int main() {
    uint8_t d = 0;

    texte(2, 2, "DROITE POUR AVANCER");

    for (uint8_t x = 0; x < 32; x++) {
        poser(x, 14, MUR);
        poser(x, 15, MUR);
    }

    while (true) {
        image();

        if (bouton(DROITE)) d++;
        if (bouton(GAUCHE)) d--;

        defiler(d, 0);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Le sol glisse sous l'écran
quand on pousse la croix.



l'écran, au bout de vingt images

59. La caméra : c'est le monde qui bouge

Le héros ne se déplace pas — comme dans Mario.

Dans un jeu qui défile, le personnage **ne bouge pas à l'écran**. Il reste au milieu, et c'est le décor qui passe derrière lui. Mario n'avance pas : il piétine au centre pendant qu'un monde de plusieurs écrans glisse vers la gauche.

Il y a donc **deux espaces** : l'écran, qui fait 160 pixels de large, et le monde, qui fait ce qu'on veut. `defiler(camera, 0)` dit simplement **quel morceau du monde on regarde**.

Et voici pourquoi cela tient en si peu de lignes : **les lutins ne défilent pas**. `defiler()` ne déplace que le fond ; un `sprite()` est posé sur l'écran, à la place où on l'écrit. Le héros au milieu, c'est une ligne, toujours la même : `sprite(0, 76, 112, HEROS);`

Une seule variable suffit pour les deux rôles. `camera` est la fenêtre **et** la position du héros dans le monde : il est en `camera + 76`. Le jour où un ennemi arrive, lui vit dans le monde, et c'est à lui de se convertir — `sprite(1, xEnnemi - camera, 112, ENNEMI)`.

La preuve est à l'écran, et elle est gratuite : **même la consigne s'en va**. Elle est écrite dans le décor, donc elle voyage avec lui. Le bonhomme, lui, ne bouge pas d'un pixel.

LE PROGRAMME — 1673 OCTETS

```
Tuile HEROS = {
    "00333300",
    "03222230",
    "32122123",
    "32222223",
    "03222230",
    "03333330",
    "00300300",
    "03300330",
};

Tuile SOL = {
    "33333333",
    "32323232",
    "23232323",
    "32323232",
    "23232323",
    "32323232",
    "23232323",
    "32323232",
};

Tuile ARBRE = {
    "00033000",
    "00333300",
    "03333330",
    "33333333",
    "03333330",
    "00033000",
    "00033000",
    "00033000",
};

int main() {
    uint8_t camera = 0;

    texte(1, 2, "IL NE BOUGE PAS");
    texte(1, 4, "CEST LE MONDE QUI");
    texte(1, 5, "PASSE DERRIERE LUI");

    for (uint8_t c = 0; c < 32; c++) {
        poser(c, 15, SOL);
        poser(c, 16, SOL);
        poser(c, 17, SOL);
    }

    poser(3, 14, ARBRE);
    poser(12, 14, ARBRE);
    poser(25, 14, ARBRE);

    while (true) {
        image();

        if (bouton(DROITE)) camera = camera + 1;
        if (bouton(GAUCHE)) camera = camera - 1;

        defiler(camera, 0);
        sprite(0, 76, 112, HEROS);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Le bonhomme reste planté au milieu ; les arbres, le sol et même la consigne passent derrière lui.

IL NE BOUGE PAS
CEST LE MONDE QUI
PASSE DERRIERE LUI



l'écran, au bout de vingt images

60. Trois variables qui parlent de la même chose

Une `struct`, plutôt que `bossX`, `bossY`, `bossVie`.

Sans `struct`, on écrirait `bossX`, `bossY`, `bossVie` — et le jour où il faut un second ennemi, on recopie trois variables et l'on en oublie une.

Une `struct` fait **au plus 255 octets**, et chaque champ est un octet. C'est un enregistrement, pas un objet : ni héritage, ni constructeur.

La leçon « Une struct : ce qui va ensemble » en fait un usage complet ; celle-ci n'en montre que la forme.

LE PROGRAMME — 1332 OCTETS

```
struct Ennemi {
    uint8_t x;
    uint8_t y;
    uint8_t vie;
};

Ennemi boss;

int main() {
    boss.x = 40;
    boss.y = 60;
    boss.vie = 3;

    texte(2, 4, "X");    nombre(6, 4, boss.x);
    texte(2, 6, "VIE");  nombre(6, 6, boss.vie, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« X 040 » et « VIE 3 » : les deux champs de la même struct, relus.

X 040
VIE 3

l'écran, au bout de vingt images

61. Un tableau de « struct »

Trois ennemis, c'est UNE ligne de plus — pas neuf variables.

`troupe[i].vie` : l'index est calculé à l'exécution, le décalage du champ est connu à la compilation. **Le compilateur fait le calcul d'adresse** ; nous, nous écrivons ce que nous voulons dire.

C'est ici que la `struct` prend son sens. Passer de un à trois ennemis coûte une ligne ; sans elle, il aurait fallu six variables de plus et six lignes à tenir d'accord.

LE PROGRAMME — 1490 OCTETS

```
struct Ennemi {
    uint8_t x, y, vie;
};

Ennemi troupe[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) {
        troupe[i].x = 20 + i * 40;
        troupe[i].y = 60;
        troupe[i].vie = 3;
    }

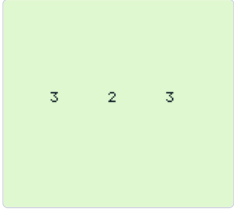
    troupe[1].vie--;

    for (uint8_t i = 0; i < 3; i++) nombre(4 + i * 5, 8, troupe[i].vie, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« 3 2 3 » — seul celui du milieu a été touché.



3 2 3

l'écran, au bout de vingt images

62. Une méthode : la fonction qui connaît son objet

``troupe[i].avancer()`` plutôt que ``avancer(i)``.

Une méthode s'écrit dans la `struct`, et travaille sur l'objet devant le point. Dans son corps, les champs s'écrivent sous leur nom nu : `x++`, et non `this->x++`.

Ce qu'elle coûte, exactement : l'adresse de l'objet est rangée dans **deux octets réservés à cette méthode**, puis c'est le même `call` qu'ailleurs. Pas de table de fonctions, pas de pile d'objets — et le compilateur compte ces deux octets comme les autres.

Ce qui reste refusé, et pourquoi : `class` (le message propose `struct`), `virtual` (il faudrait une table de fonctions et un appel indirect), l'héritage (il déplacerait les décalages des champs), les constructeurs (du code implicite à la déclaration, alors que les globales prennent leur valeur avant `main`). Et une méthode **ne prend pas d'argument** : elle reçoit déjà son objet.

LE PROGRAMME — 1595 OCTETS

```
struct Ennemi {
    uint8_t x, y, vie;

    void avancer() { x++; }
    uint8_t vivant() { return vie > 0; }
};

Ennemi troupe[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) { troupe[i].x = 20 + i * 40; troupe[i].vie = 3; }
    troupe[1].vie = 0;

    while (true) {
        for (uint8_t i = 0; i < 3; i++)
            if (troupe[i].vivant()) troupe[i].avancer();

        for (uint8_t i = 0; i < 3; i++) nombre(3 + i * 6, 8, troupe[i].x);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Deux nombres qui montent, et celui du milieu immobile.

025 060 104

l'écran, au bout de vingt images

63. Les trois façons de faire la même chose

À la suite, par une fonction, par une méthode — et ce que chacune coûte.

À la suite, tout est dans `main` : `if (ennemiVie > 0) ennemiX++;`, avec deux variables globales. C'est le plus court, et pour **un** ennemi c'est le bon choix. Le jour où il en faut trois, il faut six variables et trois fois les mêmes lignes.

Par des fonctions : les données sont rangées dans un tableau de `struct`, le geste est nommé, et l'objet se désigne par son index — `avancer(i)`, avec `void` `avancer(uint8_t i) { if (troupe[i].vie > 0) troupe[i].x++; }`. Coût : un octet par argument.

Par des méthodes, ci-dessous : rigoureusement le même travail, mais le geste est écrit **là où vivent les données**, et l'objet ne se passe plus. Coût : deux octets par méthode.

Mesuré, sur ce programme : **1344 octets** à la suite, **1590** par fonctions, **1610** par méthodes. Aucune n'est meilleure dans l'absolu — la troisième rend en lisibilité ce qu'elle prend en octets, et c'est bien pour cela que le compilateur les compte et les annonce.

LE PROGRAMME — 1639 OCTETS

```
struct Ennemi {
    uint8_t x, vie;

    void avancer() { if (vie > 0) x++; }
    void blesser() { if (vie > 0) vie--; }
};

Ennemi troupe[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) { troupe[i].x = 20 + i * 40; troupe[i].vie = 3; }
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        for (uint8_t i = 0; i < 3; i++) {
            troupe[i].avancer();
            if (appui && !avant) troupe[i].blesser();
        }
        avant = appui;

        for (uint8_t i = 0; i < 3; i++) nombre(2 + i * 6, 8, troupe[i].x);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois nombres qui montent ; A les blesse tous les trois.

025 064 104

l'écran, au bout de vingt images

64. Une méthode qui en appelle une autre

Pourquoi chaque méthode a **SES** deux octets, et non deux partagés.

Dans `blesser()`, l'appel nu `vivant()` veut dire « **la mienne** » : l'objet en cours est passé tel quel, comme en C++.

Et voici pourquoi chaque méthode a **ses** deux octets plutôt que deux partagés par la `struct` : si `avancer()` appelait `autre.vivant()`, un emplacement commun serait écrasé, et `avancer()` reprendrait son travail sur le mauvais ennemi. Le partage aurait marché sur les exemples simples et échoué exactement là où c'est le plus dur à voir.

La règle du non-appel-à-soi-même vaut aussi ici : **une méthode ne peut pas s'appeler elle-même**, ni par un détour. Le compilateur nomme le cycle, méthode par méthode.

LE PROGRAMME — 1412 OCTETS

```
struct Ennemi {
    uint8_t x, vie;

    uint8_t vivant() { return vie > 0; }
    void blesser()   { if (vivant()) vie--; }
};

Ennemi troupe[2];

int main() {
    troupe[0].vie = 3;
    troupe[1].vie = 0;

    troupe[0].blesser();
    troupe[1].blesser();

    texte(2, 6, "VIVANT MORT");
    nombre(4, 8, troupe[0].vie, 1);
    nombre(9, 8, troupe[1].vie, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« 2 » et « 0 » — le mort
n'est pas descendu à 255.

V I V A N T M O R T
2 0

l'écran, au bout de vingt images

65. Passer d'un écran à l'autre

Un titre, une partie, une fin.

Le squelette de n'importe quel jeu. Deux règles, et il n'y en a pas d'autres.

Une variable dit où l'on est. Un `enum` lui donne ses valeurs possibles, et leur donne des noms : `TITRE`, `JEU`, `FIN`. Le nom de l'`enum` devient un type d'un octet, et `Scene scene` se lit mieux que `uint8_t scene`.

Un seul endroit change d'écran. `allerA()` éteint, efface, redessine, rallume — toujours dans cet ordre. Si chaque bouton faisait son propre ménage, l'un finirait par oublier d'effacer, et deux écrans se mélangeraient.

Le `switch` aiguille sur la scène. Il fonctionne comme en C++, `break` **compris** : sans lui, l'exécution tombe dans le cas suivant. C'est un piège célèbre, mais c'est le sens du langage, et le trahir en douce serait pire.

On éteint l'écran pour effacer parce que repeindre toute la carte demande bien plus de temps qu'une image n'en contient. `effacer()` sans argument vide les 32×32 de la carte d'un coup — cette leçon écrivait autrefois ses deux boucles à la main, et n'effaçait alors que les 20×18 visibles : le décor qui défilait ramenait l'écran d'avant.

LE PROGRAMME — 1582 OCTETS

```
enum Scene { TITRE, JEU, FIN };

Scene scene = TITRE;

void allerA(Scene ou) {
    scene = ou;

    ecran(0);
    effacer();

    switch (scene) {
        case TITRE:
            texte(6, 5, "MON JEU");
            texte(4, 10, "A POUR JOUER");
            break;

        case JEU:
            texte(5, 4, "C EST PARTI");
            texte(4, 10, "A POUR FINIR");
            break;

        case FIN:
            texte(7, 5, "FINI");
            texte(3, 10, "A POUR REVENIR");
            break;
    }

    ecran(1);
}

int main() {
    uint8_t aAvant = 0;

    allerA(TITRE);

    while (true) {
        image();

        uint8_t a = bouton(A);

        if (a && !aAvant) {
            if (scene == TITRE) allerA(JEU);
            else if (scene == JEU) allerA(FIN);
            else allerA(TITRE);
        }

        aAvant = a;
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

« MON JEU », puis A change d'écran à chaque appui.

MON JEU

A POUR JOUER

l'écran, au bout de vingt images

66. La pesanteur et le saut

Tomber, et retomber.

Le processeur ne connaît **pas les nombres négatifs** : `0 - 1` vaut 255. Une vitesse qui doit monter *et* descendre ne peut donc pas s'écrire directement.

On la décale : `ZEROV` vaut 16, et la vitesse vit autour. **Moins de 16, on monte ; plus de 16, on tombe.** Sans ce décalage, un saut deviendrait une chute de 240 pixels par image.

`const uint8_t ZEROV = 16;` ne coûte rien : la valeur est connue à la compilation, et le compilateur l'écrit directement dans le code. Un `const` n'occupe pas d'octet de mémoire.

La pesanteur ajoute 1 à chaque image. Sauter, c'est poser une vitesse bien en dessous de 16 : elle remonte d'elle-même, le mouvement s'inverse, et la courbe est celle d'un vrai saut.

LE PROGRAMME — 1547 OCTETS

```
Tuile BALLE = {
    "00333300",
    "03222230",
    "32222223",
    "32222223",
    "32222223",
    "32222223",
    "03222230",
    "00333300",
};

const uint8_t ZEROV = 16;
const uint8_t SOL = 120;

int main() {
    uint8_t x = 76;
    uint8_t y = SOL;
    uint8_t vy = ZEROV;
    uint8_t auSol = 1;
    uint8_t aAvant = 0;

    texte(4, 2, "A POUR SAUTER");

    while (true) {
        image();

        uint8_t a = bouton(A);

        if (a && !aAvant && auSol) {
            vy = ZEROV - 7;
            auSol = 0;
        }

        aAvant = a;

        vy++;
        if (vy > ZEROV + 4) vy = ZEROV + 4;

        if (vy < ZEROV) y = y - ZEROV + vy;
        if (vy > ZEROV) y = y + vy - ZEROV;

        if (y >= SOL) {
            y = SOL;
            vy = ZEROV;
            auSol = 1;
        }

        sprite(0, x, y, BALLE);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

La boule saute, ralentit en l'air, et retombe sur le sol.

A POUR SAUTER



l'écran, au bout de vingt images

67. Le verre qui se brise

Quatre éclats, et trois variables.

La chute est celle de la leçon du même nom : les pixels dans `y`, les seizièmes dans `frac`, la pesanteur qui s'ajoute à la vitesse. Ce qui change ici, c'est **ce qui arrive au contact**.

Un objet qui se brise ne disparaît pas : il est **remplacé par ses morceaux**. Au choc, le verre part se cacher et quatre éclats prennent sa place, exactement à l'endroit où il s'est posé.

Cacher un lutin ne demande aucune fonction : on l'écrit à 160, hors des 144 lignes de l'écran. Il existe toujours — les quarante existent toujours — on choisit simplement de ne pas le voir. C'est pour cela que les cinq `sprite()` de la fin sont écrits une fois pour toutes, sans un seul `if` : ce sont les variables qui décident qui est visible.

On ne simule pas quatre éclats. Une image plus tôt ils étaient un seul objet : ils partent donc ensemble, à la même hauteur, et ne diffèrent que par ce qu'ils prennent de côté. Une seule hauteur `ey`, un seul compteur `t`, et les quatre positions se calculent — `76 - 3 * t`, `76 - t`, `76 + t`, `76 + 3 * t`. **Trois variables au lieu de douze**.

Leur envol est la vitesse décalée du saut : `ZEROV - 6` les projette vers le haut, la pesanteur les rattrape. Les deux éclats rapides retombent trois fois plus loin que les lents, sans qu'on ait rien à écrire pour ça — même temps de vol, vitesse triple.

`t` n'avance **que tant qu'ils volent**. Posés, il s'arrête, et les éclats restent exactement où ils sont tombés. C'est cette ligne-là, et aucune autre, qui fait la différence entre du verre brisé et quatre morceaux qui glissent à l'infini.

```

Tuile VERRE = {
    "00333300",
    "03211230",
    "32112223",
    "32122223",
    "32222223",
    "32222223",
    "03222230",
    "00333300",
};

Tuile ECLAT = {
    "00000000",
    "00000300",
    "00003130",
    "00031230",
    "00312300",
    "03123000",
    "03330000",
    "00000000",
};

const uint8_t CIEL = 8;
const uint8_t SOL = 128;
const uint8_t CACHE = 160; // hors de l'écran : un lutin écrit là ne se voit pas
const uint8_t G = 2;
const uint8_t ZEROV = 16; // la vitesse d'un éclat, décalée : 16 est l'immobilité
const uint8_t ATTENTE = 60;

int main() {
    uint8_t y = CIEL;
    uint8_t frac = 0;
    uint8_t vy = 0;

    uint8_t brise = 0;
    uint8_t t = 0; // images de vol - c'est lui qui écarte les éclats
    uint8_t ey = CACHE; // leur hauteur, la même pour les quatre
    uint8_t evy = ZEROV;
    uint8_t pose = 0;

    texte(0, 2, "IL TOMBE ET IL CASSE");
    texte(0, 17, "#####");

    while (true) {
        image();

        if (brise) {
            evy = evy + 1;
            if (evy < ZEROV) ey = ey - ZEROV + evy;
            if (evy > ZEROV) ey = ey + evy - ZEROV;

            if (ey >= SOL) {
                ey = SOL;
                evy = ZEROV;
            } else {
                t = t + 1;
            }

            pose = pose + 1;
            if (pose > ATTENTE) {
                brise = 0;
                pose = 0;
                t = 0;
                ey = CACHE;
                y = CIEL;
                frac = 0;
                vy = 0;
            }
        } else {
            vy = vy + G;
            frac = frac + vy;
            y = y + (frac >> 4);
            frac = frac & 15;

            if (y >= SOL) {
                brise = 1;
                y = CACHE;
                ey = SOL;
                evy = ZEROV - 6;
            }
        }

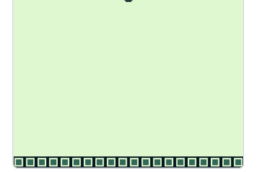
        sprite(0, 76, y, VERRE);
        sprite(1, 76 - 3 * t, ey, ECLAT);
        sprite(2, 76 - t, ey, ECLAT);
        sprite(3, 76 + t, ey, ECLAT);
        sprite(4, 76 + 3 * t, ey, ECLAT);
    }

    return 0;
}

```

Le verre tombe, éclate en quatre morceaux qui s'envolent et retombent éparpillés — puis tout recommence.

IL TOMBE ET IL CASSE



l'écran, au bout de vingt images

68. Un état de jeu

Quelques scènes, et un `switch` qui dit laquelle est en cours.

C'est presque toujours la forme d'un jeu : un `enum` de scènes, et un `switch`. Le reste — les ennemis, le décor — se branche dans le `case JEU`.

Le front est calculé **une fois**, en haut de la boucle, et servi à tous les cas. Le recalculer dans chaque `case` marcherait aussi — et un jour l'un des trois serait oublié.

Le score n'est écrit **que lorsqu'il change**, et l'étiquette une seule fois en entrant. Les réécrire à chaque image alourdit la boucle au point qu'elle déborde — et un `depuis` qui compte les tours avance alors trois fois moins vite que le temps réel.

LE PROGRAMME — 1558 OCTETS

```
enum Scene { TITRE, JEU, PERDU };

Scene ou = TITRE;
uint8_t score = 0;
uint8_t avant = 0;
uint8_t depuis = 0;

int main() {
    while (true) {
        uint8_t appui = bouton(START);
        uint8_t front = appui && !avant;
        avant = appui;

        switch (ou) {
            case TITRE:
                texte(4, 8, "APPUIE START");
                if (front) {
                    texte(4, 8, " ");
                    score = 0; depuis = 0; ou = JEU;
                    texte(2, 4, "SCORE"); // une fois, en entrant
                    nombre(9, 4, score);
                }
                break;

            case JEU:
                depuis++;
                if (depuis >= 30) { depuis = 0; score++; nombre(9, 4, score); }
                if (score >= 20) ou = PERDU;
                break;

            case PERDU:
                texte(5, 8, "PERDU");
                if (front) { texte(5, 8, " "); ou = TITRE; }
                break;
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

START lance la partie, le score monte, et à 20 c'est perdu.

APPUIE START

l'écran, au bout de vingt images

69. Le score, et le panneau

Une seconde couche, qui ne défile pas avec le décor.

Le **panneau** est une couche posée par-dessus le fond. Il ne défile pas : c'est exactement ce qu'on veut d'un score pendant que le décor glisse.

`panneau(x, y)` le place et l'allume, `cachePanneau()` l'ôte sans rien perdre de son contenu.

Deux détails qui se paient cher si on les oublie. Le compteur `depuis` plutôt qu'un multiple de `images()` : le corps de cette boucle écrit dans le panneau, il déborde d'une image, et le multiple serait enjambé — **le score resterait bloqué à zéro sans que rien ne le dise**. Et `nombrePanneau()` n'est appelé **que quand le score change** : le réécrire à chaque tour alourdit la boucle pour rien.

LE PROGRAMME — 1368 OCTETS

```
int main() {
    uint8_t score = 0;
    uint8_t depuis = 0;
    uint8_t glisse = 0;

    panneau(0, 0);
    textePanneau(1, 0, "SCORE");

    while (true) {
        depuis++;
        if (depuis >= 20) { depuis = 0; score++; nombrePanneau(8, 0, score); }

        glisse++;
        defiler(glisse, 0);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le score monte dans le panneau pendant que le fond glisse.

SCORE

l'écran, au bout de vingt images

70. Des ennemis qui vivent et qui meurent

Les méthodes, dans un vrai tour de boucle.

`hasard()` lit le compteur libre du matériel : un nombre imprévisible, sans table ni graine. `semer(n)` permet de choisir son point de départ quand on veut au contraire une suite reproductible.

`cacher(n)` ôte le lutin de l'écran. Sans lui, un ennemi mort resterait affiché, immobile — ce qui se remarque tout de suite, heureusement.

LE PROGRAMME — 1782 OCTETS

```
Tuile ENNEMI = {
    "#####.", "#-#-#-#", "#####", "#.###.#",
    "#####", ".#.#.#.", "#..#.#", ".#...#.",
};

struct Monstre {
    uint8_t x, y, vie;

    uint8_t vivant() { return vie > 0; }
    void avancer() { if (vivant()) x++; }
    void tuer() { vie = 0; }
};

Monstre troupe[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) {
        troupe[i].x = 10 + i * 30;
        troupe[i].y = 40 + i * 24;
        troupe[i].vie = 1;
    }

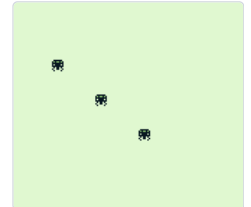
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        if (appui && !avant) troupe[hasard() % 3].tuer();
        avant = appui;

        for (uint8_t i = 0; i < 3; i++) {
            troupe[i].avancer();
            if (troupe[i].vivant()) sprite(i, troupe[i].x, troupe[i].y, ENNEMI);
            else cacher(i);
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois monstres qui avancent ; A en fait disparaître un au hasard.



l'écran, au bout de vingt images

La même chose, à la souris

 + tuile 8 × 8 + perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus. [Passer en couleur](#)

nuance 3
Nuances 

ENNEMI tuile n° 44 — poser(x, y, ENNEMI)

– zoom 22× + zoom

🖥️ Plein écran

🖌️ Peindre ✓

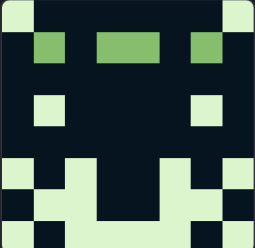
📏 Sélectionner et déplacer

📄 Copier le dessin

📌 Coller ici

📄 Dupliquer

🗑️ Effacer la tuile



 taille réelle — 8 × 8 pixels

71. Les collisions

Regarder **AVANT** de bouger, et non reculer après.

`lire(colonne, ligne)` rend le numéro de la tuile affichée. Comparer à `MUR` suffit : **le nom d'une tuile EST son numéro**.

La forme compte. On calcule la position **voulue**, on regarde ce qu'il y a, et l'on n'y va que si c'est libre. Bouger puis reculer marche aussi, mais laisse une image où le personnage est dans le mur — et cette image finit toujours par se voir.

LE PROGRAMME — 1665 OCTETS

```
Tuile MUR = {
    "#####", "#-----#", "#-----#", "#-----#",
    "#-----#", "#-----#", "#-----#", "#####",
};

int main() {
    for (uint8_t y = 0; y < 18; y++) { poser(0, y, MUR); poser(19, y, MUR); }
    for (uint8_t x = 0; x < 20; x++) { poser(x, 0, MUR); poser(x, 17, MUR); }
    poser(10, 8, MUR);

    uint8_t c = 5;
    uint8_t l = 8;

    while (true) {
        uint8_t vc = c;
        uint8_t vl = l;

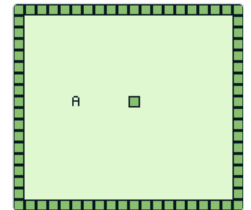
        if (bouton(DROITE)) vc++;
        if (bouton(GAUCHE)) vc--;
        if (bouton(BAS)) vl++;
        if (bouton(HAUT)) vl--;

        /* On regarde AVANT de bouger : la case visée est-elle libre ? */
        if (lire(vc, vl) != MUR) { c = vc; l = vl; }

        sprite(0, c * 8, l * 8, 1);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le curseur s'arrête net contre les murs et contre le bloc isolé.



l'écran, au bout de vingt images

La même chose, à la souris

MUR

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus. [Passer en couleur](#)

nuance 3

Nuances 

MUR

tuile n° 44 — poser(x, y, MU

– zoom 22×

+ zoom

Plein écran

Peindre ✓

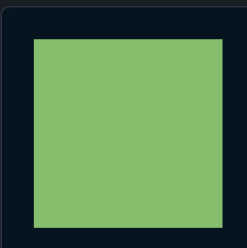
Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

72. Ralentir sans compter les images à la main

``retard()`` dit quand le tour de boucle a débordé.

`retard()` rend 1 si le tour précédent a duré **plus d'une image**. C'est la seule façon honnête de savoir qu'un jeu ne tient plus les soixante images par seconde : le compteur d'images de la page mesure le navigateur, pas la console.

Redessiner trois cent soixante tuiles à chaque tour est un bon moyen de faire monter ce compteur. **Un vrai jeu ne redessine que ce qui a changé** — et c'est exactement le débordement qui fait rater un `images() % 20 == 0`.

LE PROGRAMME — 1413 OCTETS

```
int main() {
    uint8_t sautees = 0;

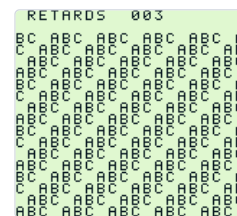
    while (true) {
        /* Un travail volontairement lourd, pour voir la mesure bouger. La
           première ligne est laissée au compte rendu : la redessiner l'effacerait
           aussitôt, et l'on croirait que texte() ne marche pas. */
        for (uint8_t y = 2; y < 18; y++)
            for (uint8_t x = 0; x < 20; x++)
                poser(x, y, (x + y) % 4);

        if (retard()) sautees++;

        texte(1, 0, "RETARDS");
        nombre(10, 0, sautees);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le compteur de retards monte : la boucle ne tient pas l'image.



l'écran, au bout de vingt images

73. Passer à la salle suivante, sans bouton

Ce n'est pas un bouton qui change de salle : c'est le personnage qui sort du cadre.

La leçon « Passer d'un écran à l'autre » changeait de scène **sur un appui**. Ici, rien de tel : c'est la POSITION du héros qui décide — sortir par la droite fait apparaître la salle suivante, avec le héros collé au bord gauche.

La même règle que `allerA()` s'applique : `ecran(0)`, `effacer()`, redessiner, puis `ecran(1)` — toujours dans cet ordre, pour ne pas voir l'ancienne salle se mélanger à la nouvelle pendant qu'on l'efface.

Le héros ne s'arrête jamais à un bord comme dans « Le garder dans l'écran » : ici, dépasser le bord est justement ce qui déclenche le changement. Les deux techniques répondent à deux besoins opposés, avec le même test `x > DROITE_MAX`.

LE PROGRAMME — 1671 OCTETS

```
Tuile HEROS = {
    "..####.", ".#-##-#.", "#####", "#.####.#",
    "#####", "..#.#..", ".#...#.", "##...##",
};

const uint8_t GAUCHE_MAX = 8;
const uint8_t DROITE_MAX = 152;

uint8_t salle = 0;

void dessinerSalle() {
    ecran(0);
    effacer();

    if (salle == 0) {
        texte(6, 1, "SALLE A");
        poser(18, 8, 1);
    } else {
        texte(6, 1, "SALLE B");
        poser(1, 8, 1);
    }

    ecran(1);
}

int main() {
    uint8_t x = 76;
    uint8_t y = 68;

    dessinerSalle();

    while (true) {
        if (bouton(GAUCHE)) x--;
        if (bouton(DROITE)) x++;
        if (bouton(HAUT)) y--;
        if (bouton(BAS)) y++;

        if (x > DROITE_MAX && salle == 0) {
            salle = 1;
            x = GAUCHE_MAX;
            dessinerSalle();
        } else if (x < GAUCHE_MAX && salle == 1) {
            salle = 0;
            x = DROITE_MAX;
            dessinerSalle();
        }

        sprite(0, x, y, HEROS);
        image();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

« SALLE A », puis « SALLE B » dès que le héros sort par la droite — il réapparaît collé au bord gauche.



l'écran, au bout de vingt images

74. Se souvenir d'une partie à l'autre

Un meilleur score qui survit à l'extinction.

La cartouche porte une pile et une petite mémoire. `sauver(numero, valeur)` y écrit, `sauvegarde(numero)` y relit — 256 cases, et elles sont encore là après avoir éteint la console.

Cette mémoire n'est pas ouverte en permanence : il faut la **déverrouiller** avant d'y toucher et la refermer aussitôt. Le compilateur pose le verrou lui-même à chaque accès. Ce n'est pas de la prudence excessive : laissée ouverte, une coupure de courant au mauvais moment la corrompt.

À la toute première partie, cette mémoire contient n'importe quoi. Un jeu y écrit donc **une marque à lui** et ne fait confiance au reste que s'il la retrouve. C'est ce que fait la première ligne de `main()`.

LE PROGRAMME — 1582 OCTETS

```
const uint8_t MARQUE = 42;

uint8_t meilleur = 0;
uint8_t score = 0;
uint8_t attente = 0;

int main() {
    // La cartouche a-t-elle déjà servi ?
    if (sauvegarde(0) != MARQUE) {
        sauver(0, MARQUE);
        sauver(1, 0);
    }

    meilleur = sauvegarde(1);

    texte(2, 4, "SCORE");
    texte(2, 8, "RECORD");

    while (true) {
        image();

        attente++;

        if (attente > 30) {
            attente = 0;

            if (score < 9) score++;

            if (score > meilleur) {
                meilleur = score;
                sauver(1, meilleur);
            }
        }

        poser(10, 4, 27 + score);
        poser(10, 8, 27 + meilleur);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Le score monte, et le record le suit — puis lui survit.

SCORE 0

RECORD 0

l'écran, au bout de vingt images

75. La musique d'un jeu

Tout ensemble : un lutin qu'on déplace, une pièce à attraper, et un air qui ne s'arrête jamais.

Voici les dix niveaux réunis dans un seul programme : des tuiles dessinées, des lutins au pixel près, des boutons lus à chaque image, un score — et **deux airs**.

`jouer(1, MUSIQUE, 8, 1)` lance la musique de fond ; le quatrième argument, `1`, la fait **recommencer sans fin**. Ensuite le programme n'a plus rien à tenir : l'air avance tout seul dans l'interruption de la console, soixante fois par seconde, **même quand le jeu est en retard**.

Attraper la pièce lance `jouer(1, VICTOIRE, 5)` : la fanfare **prend la voix 1**, par-dessus la musique, exactement comme dans un vrai jeu. Elle, elle ne boucle pas.

`airFin(1)` dit quand la fanfare est arrivée au bout. C'est là qu'on relance la musique de fond — sans cela, la voix resterait muette après la victoire, et l'on chercherait longtemps pourquoi.

La **partition ci-dessous** écrit les deux airs à la souris, et « Écouter » les fait entendre sans compiler. Change une note : le programme se réécrit, la cartouche se refait, et la console joue ta version dans la seconde.

LE PROGRAMME — 2132 OCTETS

```
/* La musique tourne pendant que le jeu tourne. */

Air MUSIQUE = {
  "D04 10", "==", "SOL4 10", "==", "MI4 10", "==", "SOL4 10", "==",
  "FA4 10", "==", "LA4 10", "==", "SOL4 10", "==", "--", "--",
};

Air VICTOIRE = {
  "D05 14", "MI5 14", "SOL5 15", "==", "--",
};

Tuile HEROS = {
  "00333300",
  "03222330",
  "32122123",
  "32222223",
  "33222233",
  "03333330",
  "00300300",
  "03300330",
};

Tuile PIECE = {
  "00033000",
  "00311300",
  "03133130",
  "03133130",
  "03133130",
  "00311300",
  "00033000",
  "00000000",
};

uint8_t x = 40;
uint8_t piece = 60;
uint8_t score = 0;

int main() {
  texte(1, 1, "SCORE");
  jouer(1, MUSIQUE, 8, 1);

  while (true) {
    image();

    if (bouton(DROITE) && x < 152) x++;
    if (bouton(GAUCHE) && x > 8) x--;

    sprite(0, x, 80, HEROS);
    sprite(1, piece, 80, PIECE);

    /* Attrapée ? On avance le score, on joue la fanfare par-dessus la
       musique, et la pièce repart plus loin. */
    if (x + 6 > piece && piece + 6 > x) {
      if (score < 9) score++;
      poser(7, 1, 27 + score);
      jouer(1, VICTOIRE, 5);
      piece = piece + 40;
      if (piece > 150) piece = 20;
    }

    /* La fanfare finie, la musique reprend là où elle en était : au début. */
    if (airFin(1)) jouer(1, MUSIQUE, 8, 1);
  }

  return 0;
}
```

CE QU'ON DOIT VOIR

DROITE et GAUCHE déplacent le héros. La pièce attrapée fait monter le score et sonner une fanfare.

SCORE



l'écran, au bout de vingt images

La même chose, à la souris

MUSIQUE

VICTOIRE

+ air

MUSIQUE

▶ Écouter

🎵 note

- silence

= tenir

▼

▲

- pas

+ pas

8

images par pas

`jouer(1, MUSIQUE, 8);` — le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas.
Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

76. Le monde glisse, le héros ne bouge pas

La position dans le monde vaut « défilement + 76 ».

Le lutin ne bouge **jamais** : il est à 76 pixels, image après image. Ce qui change, c'est `defiler()`.

C'est ce qui distingue un écran fixe d'un monde. La position du personnage **dans le monde** vaut `monde + 76`, et c'est ce nombre-là qu'on compare au décor — pas celui du lutin, qui est une constante.

La leçon « La caméra : c'est le monde qui bouge » part du même principe et le pousse jusqu'au bord du niveau.

LE PROGRAMME — 1456 OCTETS

```
Tuile SOL = {
    "#####", "-----", "-----", "-----",
    "-----", "-----", "-----", "-----",
};

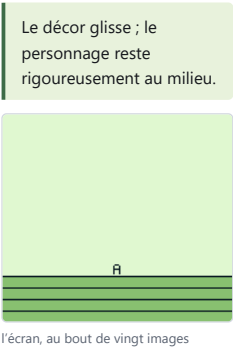
int main() {
    for (uint8_t x = 0; x < 20; x++)
        for (uint8_t y = 14; y < 18; y++)
            poser(x, y, SOL);

    uint8_t monde = 0;

    while (true) {
        if (bouton(DROITE)) monde++;
        if (bouton(GAUCHE)) monde--;

        defiler(monde, 0);
        sprite(0, 76, 104, 1);
        image();
    }
}
```

CE QU'ON DOIT VOIR



La même chose, à la souris

 + tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus. [Passer en couleur](#)

nuance 3

Nuances

SOL

tuile n° 44 — poser(x, y, SO

– zoom22×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile



taille réelle — 8 × 8 pixels

77. Un record qui survit à l'extinction

La cartouche a une pile, et de la mémoire qui reste.

`sauver(case, valeur)` et `sauvegarde(case)` écrivent et relisent la **mémoire sauvegardée de la cartouche** — celle qui survit à l'extinction.

Sur une vraie console, c'est une pile qui la tient. Dans la page, c'est le navigateur ; le programme, lui, ne voit aucune différence.

On ne sauve **que lorsque le record change**. Écrire à chaque image userait la mémoire d'une vraie cartouche, et n'apporterait rien.

LE PROGRAMME — 1462 OCTETS

```
int main() {
    uint8_t record = sauvegarde(0);
    uint8_t score = 0;
    uint8_t avant = 0;

    texte(1, 2, "RECORD");
    nombre(10, 2, record);

    while (true) {
        uint8_t appui = bouton(A);
        if (appui && !avant) score++;
        avant = appui;

        texte(1, 6, "SCORE");
        nombre(10, 6, score);

        if (score > record) {
            record = score;
            sauver(0, record);
            nombre(10, 2, record);
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le record suit le score dès qu'il le dépasse.

RECORD 000

SCORE 000

l'écran, au bout de vingt images

78. Compter ce que coûte une boucle

Le compilateur ne cache pas ses prix.

Compiler donne son compte rendu : octets de programme, fonctions, variables, mémoire de travail. **Changer une ligne et recompiler montre ce qu'elle a coûté** — c'est la seule mesure qui ne mente pas.

Deux choses à savoir, mesurées et non supposées. `100 * x` et `x * 100` ne coûtent pas la même chose : la routine générale boucle sur son opérande de droite, alors le compilateur met le nombre connu à droite quand l'opérateur s'en moque.

Et `x / 16` devient quatre décalages, `x % 16` un `and` — **quand le diviseur est connu**. Sinon, c'est la routine générale, et elle se paie.

LE PROGRAMME — 1361 OCTETS

```
const uint8_t TABLE[] = { 3, 1, 4, 1, 5, 9, 2, 6 };

uint8_t total() {
    uint8_t somme = 0;
    for (uint8_t i = 0; i < sizeof(TABLE); i++) somme += TABLE[i];
    return somme;
}

int main() {
    texte(2, 6, "TOTAL");
    nombre(10, 6, total());

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

031 — la somme de la table gravée.

TOTAL 031

l'écran, au bout de vingt images

79. Le même jeu, deux fois

Une balle à la suite, trois balles par méthodes — le corps est le même.

À la suite, une balle demande quatre variables et huit lignes dans `main` : `if (versDroite) { x += 2; if (x > 150) versDroite = 0; }`, et ainsi de suite. Pour **une** balle, c'est parfait — 1427 octets.

Ci-dessous, trois balles **par méthodes**. Le corps de `avancer()` est **mot pour mot** celui de la version à la suite ; ce qui a changé, c'est qu'il est écrit une fois pour trois, et rangé avec les données sur lesquelles il travaille. 1956 octets, et quinze octets de mémoire.

La seule règle honnête : écrire à la suite tant que c'est un objet, nommer par une fonction quand le geste se répète, ranger dans une méthode quand les gestes et les données vont manifestement ensemble. Et ne pas choisir d'avance — recompiler, et lire le compte rendu.

LE PROGRAMME — 1985 OCTETS

```
Tuile BALLE = {
    ".....", ".....", ".....", ".....",
    ".....", ".....", ".....", ".....",
};

struct Balle {
    uint8_t x, y, versDroite, versBas, pas;

    void avancer() {
        if (versDroite) { x += pas; if (x > 150) versDroite = 0; }
        else { x -= pas; if (x < 10) versDroite = 1; }

        if (versBas) { y += pas; if (y > 130) versBas = 0; }
        else { y -= pas; if (y < 10) versBas = 1; }
    }
};

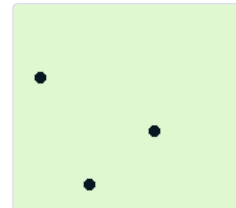
Balle balles[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) {
        balles[i].x = 20 + i * 40;
        balles[i].y = 30 + i * 20;
        balles[i].versDroite = i % 2;
        balles[i].versBas = 1;
        balles[i].pas = 1 + i;
    }

    while (true) {
        for (uint8_t i = 0; i < 3; i++) {
            balles[i].avancer();
            sprite(i, balles[i].x, balles[i].y, BALLE);
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois balles qui rebondissent, à trois vitesses différentes.



l'écran, au bout de vingt images

La même chose, à la souris

 + tuile 8 × 8

 + perso 16 × 16

 **Game Boy — 4 nuances** Les quatre nuances de la console d'origine, et rien de plus.  Passer en couleur

nuance 3

Nuances 

BALLE tuile n° 44 — poser(x, y, BALL

 zoom 22×

 zoom

 Plein écran

 Peindre ✓

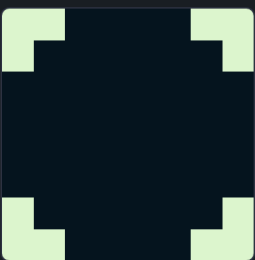
 Sélectionner et déplacer

 Copier le dessin

 Coller ici

 Dupliquer

 Effacer la tuile



 taille réelle — 8 × 8 pixels

80. Ce que le compilateur refuse, et pourquoi

Le dernier tutorial n'apprend pas à écrire : il apprend à lire les refus.

C'est la règle qui gouverne le projet. **Un long traduit en douce sur un octet, c'est un jeu qui compte faux à partir de 256 et personne n'a été prévenu** ; un refus, c'est trois secondes perdues et une ligne à réécrire.

Cinq refus qu'on rencontre vraiment. `float vitesse = 1.5;` — la machine n'a pas de virgule. `class Ennemi { ... };` — le message propose `struct`, qui fait ce qu'on voulait, méthodes comprises. `carte[3] = 1;` sur une grille — il faut **deux** index, car `carte[3]` désigne une rangée entière. `uint8_t loin(Ennemi e)` — un argument est un octet ; une `struct` se passe par son nom, visible partout, ou par son index. `TABLE[0] = 9;` sur une table `const` — elle est gravée dans la cartouche.

Chacun porte **son numéro de ligne**, et dit quoi écrire à la place. C'est ce qui fait la différence entre un compilateur qui aide et un compilateur qui punit : les deux disent non, un seul dit pourquoi. Le programme ci-dessous est la version acceptée des cinq.

LE PROGRAMME — 1438 OCTETS

```
struct Ennemi { uint8_t x, y; };
const uint8_t TABLE[] = { 1, 2, 3 };
uint8_t carte[4][5];
Ennemi boss;

/* Pas la struct : son INDEX. Un argument est un octet. */
uint8_t loin(uint8_t cible) {
    if (boss.x > cible) return boss.x - cible;
    return cible - boss.x;
}

int main() {
    boss.x = 40;
    carte[2][3] = 1;          // deux index, pas un

    uint8_t vitesse = 2;      // un octet, pas un float

    texte(1, 4, "ECART");
    nombre(9, 4, loin(10));

    texte(1, 6, "GRILLE");
    nombre(9, 6, carte[2][3], 1);

    texte(1, 8, "TABLE");
    nombre(9, 8, TABLE[2], 1);

    texte(1, 10, "PAS");
    nombre(9, 10, vitesse, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

030, 1, 3, 2 — les cinq refus, écrits comme il faut.

ECART	030
GRILLE	1
TABLE	3
PAS	2

l'écran, au bout de vingt images

81. Ce qu'une ligne de C++ devient, en vrai

« `compte++` » n'existe pas pour la machine : elle ne connaît que lire, calculer, écrire.

Cliquer sur  **MODE MACHINE** juste après avoir compilé montre ce que la cartouche contient vraiment : des instructions du processeur, pas du C++. Ce mode se souvent d'où le compilateur a rangé chaque variable et chaque fonction — un « nom retrouvé » n'existe que pour une cartouche qui vient d'être compilée ici ; une cartouche venue d'ailleurs ne montre que des nombres.

`compte = 0` devient deux instructions : `xor a` (mettre le registre A à zéro) puis `ldh [main.compte], a` (l'écrire dans la case mémoire de compte). Il n'y a pas d'instruction « mettre une variable à zéro » sur ce processeur — seulement des registres et des adresses.

`compte++` n'est **pas** une seule instruction non plus : la machine doit LIRE la valeur (`ldh a, [main.compte]`), l'AUGMENTER dans un registre (`inc a`), puis la RÉÉCRIRE (`ldh [main.compte], a`). Trois instructions pour un `++`, parce que la puce ne calcule jamais directement en mémoire — tout passe par un registre.

Le `while (true)` du programme n'existe pas non plus dans la cartouche : c'est un simple retour en arrière, `jp`, qui saute à l'instruction d'avant plutôt que de continuer. La boucle est une manière humaine de LIRE le programme ; pour la machine, ce n'est qu'un saut.

LE PROGRAMME — 1289 OCTETS

```
int main() {
    uint8_t compte = 0;

    while (true) {
        compte++;
        nombre(8, 8, compte);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un nombre qui monte à l'écran ; dans MODE MACHINE, chaque ligne de ce programme devient plusieurs instructions.

009

l'écran, au bout de vingt images

82. Sinus, cosinus, une seule table

Pas de virgule flottante, pas de type négatif : une table gravée y suffit.

Il n'y a ni `sin()` ni `cos()` ici, et pas de type signé non plus — impossible d'écrire un nombre négatif directement. La solution tient dans **une table gravée** : chaque case vaut `40 + 40 * sin(angle)`, toujours entre 0 et 80, jamais en dessous de zéro.

`cos(angle) = sin(angle + 90°)` — et 90°, c'est un quart de tour. Avec trente-deux pas pour un tour complet, un quart de tour fait huit pas. Pas besoin d'une deuxième table : `cosinus` relit la MÊME, huit cases plus loin, avec `%` pour revenir au début quand ça déborde.

Le reste est déjà connu : `aAvant` pour agir **au front** — un seul basculement par appui, et non un par image —, et deux textes de même longueur, pour que rien ne traîne de l'un à l'autre.

LE PROGRAMME — 1558 OCTETS

```
Tuile BALLE = { "..####..", ".#----#.", "#-+-+-#", "#-####-#", "#-####-#", "#-+-+-#", ".#----#.", "..####.." };

const uint8_t SINUS[] = {
    40, 48, 55, 62, 68, 73, 77, 79,
    80, 79, 77, 73, 68, 62, 55, 48,
    40, 32, 25, 18, 12, 7, 3, 1,
    0, 1, 3, 7, 12, 18, 25, 32,
};

uint8_t x = 0;
uint8_t angle = 0;
uint8_t cosinus = 0;
uint8_t aAvant = 0;

int main() {
    while (true) {
        image();

        uint8_t a = bouton(A);
        if (a && !aAvant) cosinus = !cosinus;
        aAvant = a;

        if (cosinus) texte(1, 1, "A : COSINUS");
        else texte(1, 1, "A : SINUS  ");

        uint8_t indice = cosinus ? (angle + 8) % 32 : angle;
        uint8_t y = 28 + SINUS[indice];

        sprite(0, x, y, BALLE);

        x++;
        if (x > 152) x = 0;

        angle++;
        if (angle >= sizeof(SINUS)) angle = 0;
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Une balle qui ondule en traversant l'écran ; A bascule « SINUS » en « COSINUS », et elle saute d'un quart de tour.

A : SINUS



l'écran, au bout de vingt images